Time-Varying Betas Across Sectors During Market Regimes

This notebook provides one integrated workflow for the final project. It collects sector ETF price data, merges price and macroeconomic variables, constructs daily and weekly return panels, estimates rolling CAPM betas, visualizes regime-dependent beta behavior, and tests whether rolling beta predicts next-month sector volatility.

The notebook is organized as one continuous analysis rather than separated by individual contributor sections.

```
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')

# Robust Colab / local setup.
# This notebook searches for the project folder instead of assuming one exact path.
try:
    from google.colab import drive
    drive.mount('/content/drive', force_remount=False)
except Exception:
    pass

PREFERRED_DIR = Path('/content/drive/MyDrive/5261 Final Project')
SEARCH_ROOTS = [
    PREFERRED_DIR,
    Path('/content/drive/MyDrive'),
    Path('/content'),
    Path.cwd(),
]
KEY_FILES = ['sector_etf_adj_close.csv', 'returns_clean.csv', 'macro_data.csv']

def _safe_exists(p):
    try:
        return p.exists()
    except Exception:
        return False

def find_project_dir():
    # 1. Use the preferred folder if it contains any key file.
    if _safe_exists(PREFERRED_DIR) and any(_safe_exists(PREFERRED_DIR / f) for f in KEY_FILES):
        return PREFERRED_DIR

    # 2. Search common roots for a folder containing project CSVs.
    for root in SEARCH_ROOTS:
        if not _safe_exists(root):
            continue
        for fname in KEY_FILES:
            try:
                hits = list(root.rglob(fname))
            except Exception:
                hits = []
            if hits:
                return hits[0].parent

    # 3. Fall back to the preferred location.
    return PREFERRED_DIR

DATA_DIR = find_project_dir()
OUT_DIR = DATA_DIR / 'output'
FIGURE_OUT_DIR = OUT_DIR / 'figures'
OUT_DIR.mkdir(parents=True, exist_ok=True)
FIGURE_OUT_DIR.mkdir(parents=True, exist_ok=True)

# Helper: read/write CSVs flexibly from either DATA_DIR or OUT_DIR.
def find_csv(filename):
    candidates = [DATA_DIR / filename, OUT_DIR / filename, Path('/content') / filename, Path.cwd() / filename]
    for p in candidates:
        if _safe_exists(p):
            return p
    for root in SEARCH_ROOTS:
        if not _safe_exists(root):
            continue
        try:
            hits = list(root.rglob(filename))
```

```
            except Exception:
                hits = []
            if hits:
                return hits[0]
        return None

    print(f'DATA_DIR: {DATA_DIR}')
    print(f'OUT_DIR : {OUT_DIR}')
    print('Detected CSV files:')
    for f in KEY_FILES + ['events.csv', 'returns_clean_weekly.csv']:
        print(f'  {f:<25} -> {find_csv(f)}')
```

```
Mounted at /content/drive
DATA_DIR: /content/drive/MyDrive/5261 Final Project
OUT_DIR : /content/drive/MyDrive/5261 Final Project/output
Detected CSV files:
  sector_etf_adj_close.csv  -> /content/drive/MyDrive/5261 Final Project/sector_etf_adj_close.csv
  returns_clean.csv         -> /content/drive/MyDrive/5261 Final Project/returns_clean.csv
  macro_data.csv            -> /content/drive/MyDrive/5261 Final Project/macro_data.csv
  events.csv                -> /content/drive/MyDrive/5261 Final Project/events.csv
  returns_clean_weekly.csv  -> /content/drive/MyDrive/5261 Final Project/returns_clean_weekly.csv
```

1. Yahoo Finance Price Data Collection

This section downloads adjusted closing prices for the selected sector ETFs and SPY from Yahoo Finance and saves the result as `sector_etf_adj_close.csv`. If the CSV already exists in `DATA_DIR`, the later cleaning section can read it directly.

```
# Install yfinance in Colab if needed.
try:
    import yfinance as yf
except ImportError:
    import sys
    !{sys.executable} -m pip install yfinance
    import yfinance as yf

import time
import os
import pandas as pd

SECTORS = ['XLK', 'XLE', 'XLF', 'XLV', 'XLP', 'XLU', 'XLY']
MARKET = 'SPY'
PRICE_COLS = SECTORS + [MARKET]
START_DATE = '2018-01-01'
END_DATE = '2024-12-31'

# Set to True when the price CSV needs to be downloaded or refreshed.
DOWNLOAD_YAHOO_DATA = False
PRICE_FILE = DATA_DIR / 'sector_etf_adj_close.csv'

# Optional proxy setting. Leave as None unless a local proxy is required.
PROXY_URL = None
if PROXY_URL:
    os.environ['HTTP_PROXY'] = PROXY_URL
    os.environ['HTTPS_PROXY'] = PROXY_URL
```

```
def extract_adjusted_close(downloaded_df, ticker):
    """Return one clean adjusted-close Series from a yfinance download result.

    yfinance may return either a normal single-level column index or a MultiIndex
    such as ('Adj Close', 'XLK'). This helper handles both formats.
    """
    if downloaded_df.empty:
        raise ValueError(f'No data downloaded for {ticker}')

    # Case 1: yfinance returns MultiIndex columns, usually level 0 = price field and level 1 = ticker.
    if isinstance(downloaded_df.columns, pd.MultiIndex):
        if 'Adj Close' in downloaded_df.columns.get_level_values(0):
            s = downloaded_df['Adj Close']
        elif 'Close' in downloaded_df.columns.get_level_values(0):
            # Fallback only if Adj Close is unavailable.
            s = downloaded_df['Close']
        else:
```

```
            raise ValueError(f'Neither Adj Close nor Close column found for {ticker}')

        # For a single ticker, this is often a one-column DataFrame. Convert it to Series.
        if isinstance(s, pd.DataFrame):
            if ticker in s.columns:
                s = s[ticker]
            elif s.shape[1] == 1:
                s = s.iloc[:, 0]
            else:
                raise ValueError(f'Cannot identify adjusted-close column for {ticker}')

    # Case 2: yfinance returns ordinary columns.
    else:
        if 'Adj Close' in downloaded_df.columns:
            s = downloaded_df['Adj Close']
        elif 'Close' in downloaded_df.columns:
            s = downloaded_df['Close']
        else:
            raise ValueError(f'Neither Adj Close nor Close column found for {ticker}')

    s = s.copy()
    s.name = ticker
    return s


if DOWNLOAD_YAHOO_DATA or not PRICE_FILE.exists():
    price_series = []
    for ticker in PRICE_COLS:
        print(f'Downloading: {ticker}')
        df = yf.download(
            ticker,
            start=START_DATE,
            end=END_DATE,
            auto_adjust=False,
            progress=False
        )
        price_series.append(extract_adjusted_close(df, ticker))
        time.sleep(1)

    sector_prices = pd.concat(price_series, axis=1)
    sector_prices.index.name = 'date'
    sector_prices.to_csv(PRICE_FILE)
    print(f'Saved: {PRICE_FILE}')
else:
    sector_prices = pd.read_csv(PRICE_FILE, index_col=0, parse_dates=True)
    print(f'Existing price file loaded: {PRICE_FILE}')

sector_prices.head()
```

```
Existing price file loaded: /content/drive/MyDrive/5261 Final Project/sector_etf_adj_close.csv
```

Ticker	XLK	XLE	XLF	XLV	XLP	XLU	XLY	SPY
2018-01-02	29.872919	25.747259	23.884136	72.723328	45.314568	20.132580	46.275238	236.562164
2018-01-03	30.122099	26.132856	24.012455	73.419174	45.298542	19.974422	46.487701	238.058426
2018-01-04	30.274368	26.290609	24.234875	73.523560	45.426765	19.808544	46.640114	239.061798
2018-01-05	30.592751	26.280090	24.303310	74.149803	45.627140	19.800827	47.009617	240.654953
2018-01-08	30.708111	26.437834	24.269094	73.880180	45.739338	19.985998	47.065041	241.095062

Next steps: Generate code with `sector_prices` New interactive sheet

2. Data Cleaning and Merging

This section merges sector ETF prices with macroeconomic variables and event windows, computes log returns and excess returns, attaches regime labels, and saves clean daily and weekly panels for downstream analysis.

Inputs

The notebook first tries the consolidated macro/event files. If they are missing, it falls back to raw FRED files and default event windows.

Source	Primary input	Fallback
Sector prices	`sector_etf_adj_close.csv`	generated from Yahoo Finance in Section 1
Macroeconomic variables	`macro_data.csv`, `events.csv`	`DTB3.csv`, `DGS10.csv`, `VIXCLS.csv`, `CPIAUCSL.csv` + default event list
Optional variables	`DFF.csv`, `USREC.csv`	included when present

Outputs

File	Purpose
`returns_clean.csv`	Daily panel used for descriptive statistics and realized volatility.
`returns_clean_weekly.csv`	Weekly panel used for rolling beta estimation.
`data_summary.txt`	Validation report.

Setup

```
import numpy as np
import pandas as pd
from pathlib import Path

# DATA_DIR, OUT_DIR, and FIGURE_OUT_DIR are defined in the setup cell.
OUT_DIR.mkdir(parents=True, exist_ok=True)
FIGURE_OUT_DIR.mkdir(parents=True, exist_ok=True)

SECTORS    = ['XLK', 'XLE', 'XLF', 'XLV', 'XLP', 'XLU', 'XLY']
MARKET     = 'SPY'
PRICE_COLS = SECTORS + [MARKET]

START_DATE = '2018-01-01'
END_DATE   = '2024-12-31'

# Default event windows. Used only if events.csv is not provided.
DEFAULT_EVENT_WINDOWS = [
    ('rate_hike',      '2022-03-16', '2023-07-26'),
    ('bear_2022',      '2022-01-03', '2022-10-12'),
    ('ai_rally',       '2023-10-01', '2024-12-31'),
    ('covid_recovery', '2020-03-24', '2020-12-31'),
    ('svb_crisis',     '2023-03-08', '2023-05-01'),
    ('covid_crash',    '2020-02-19', '2020-03-23'),
]

# Priority for resolving overlaps. Higher wins.
REGIME_PRIORITY = {
    'normal': 0, 'rate_hike': 1, 'bear_2022': 2, 'ai_rally': 3,
    'covid_recovery': 4, 'svb_crisis': 5, 'covid_crash': 6,
}

# Normalize the macroeconomic event naming to our internal labels.
EVENT_NAME_MAP = {
    'covid_crash': 'covid_crash', 'covid_recovery': 'covid_recovery',
    'bear_2022': 'bear_2022',     'rate_hike': 'rate_hike',
    'bank_crisis': 'svb_crisis',  'svb_crisis': 'svb_crisis',
    'ai_rally': 'ai_rally',
}
```

2. Load sector ETF prices

SPY's trading calendar becomes the master index. All other series align to these dates.

```
prices = pd.read_csv(DATA_DIR / 'sector_etf_adj_close.csv',
                     index_col=0, parse_dates=True)
prices.index.name = 'date'
prices = prices[PRICE_COLS].sort_index().loc[START_DATE:END_DATE]

print(f'Shape: {prices.shape}')
print(f'Range: {prices.index.min().date()} to {prices.index.max().date()}')
prices.head()
```

```
Shape: (1760, 8)
Range: 2018-01-02 to 2024-12-30
```

date	XLK	XLE	XLF	XLV	XLP	XLU	XLY	SPY
2018-01-02	29.872919	25.747259	23.884136	72.723328	45.314568	20.132580	46.275238	236.562164
2018-01-03	30.122099	26.132856	24.012455	73.419174	45.298542	19.974422	46.487701	238.058426
2018-01-04	30.274368	26.290609	24.234875	73.523560	45.426765	19.808544	46.640114	239.061798
2018-01-05	30.592751	26.280090	24.303310	74.149803	45.627140	19.800827	47.009617	240.654953
2018-01-08	30.708111	26.437834	24.269094	73.880180	45.739338	19.985998	47.065041	241.095062

Next steps: Generate code with `prices` New interactive sheet

3. Load macro data

Try the macroeconomic `macro_data.csv` first. If it doesn't exist, load the individual FRED files. `DFF.csv` and `USREC.csv` are optional and included when present.

```
def _load_fred_from_file(path, colname):
    df = pd.read_csv(path)
    date_col = 'observation_date' if 'observation_date' in df.columns else 'date'
    df[date_col] = pd.to_datetime(df[date_col])
    return df.rename(columns={date_col: 'date'}).set_index('date')[colname].sort_index()

raw_macro = {}

# Preferred source: macro_data.csv.
macro_path = find_csv('macro_data.csv')
returns_path = find_csv('returns_clean.csv')

if macro_path is not None:
    macro_source = f'macro_data.csv at {macro_path}'
    m = pd.read_csv(macro_path)
    # Accept either date/date-like column names.
    if 'date' not in m.columns:
        date_candidates = [c for c in m.columns if c.lower() in ['date', 'observation_date']]
        if date_candidates:
            m = m.rename(columns={date_candidates[0]: 'date'})
    m['date'] = pd.to_datetime(m['date'])
    m = m.set_index('date').sort_index()
    # Normalize common column names.
    m = m.rename(columns={
        'CPI': 'CPIAUCSL',
        'VIXCLS': 'VIX',
        '^VIX': 'VIX',
        'DFF': 'FEDFUNDS'
    })
    for col in ['DTB3', 'FEDFUNDS', 'DGS10', 'CPIAUCSL', 'VIX', 'USREC']:
        if col in m.columns:
            raw_macro[col] = pd.to_numeric(m[col], errors='coerce')

# Fallback source: returns_clean.csv already contains macro columns in your folder.
elif returns_path is not None:
    macro_source = f'existing returns_clean.csv at {returns_path}'
    m = pd.read_csv(returns_path)
    if 'date' not in m.columns:
        m = m.rename(columns={m.columns[0]: 'date'})
    m['date'] = pd.to_datetime(m['date'])
    m = m.set_index('date').sort_index()
    m = m.rename(columns={'VIXCLS': 'VIX', 'CPI': 'CPIAUCSL', 'DFF': 'FEDFUNDS'})
    for col in ['DTB3', 'FEDFUNDS', 'DGS10', 'CPIAUCSL', 'VIX', 'USREC']:
        if col in m.columns:
            raw_macro[col] = pd.to_numeric(m[col], errors='coerce')

# Last fallback: individual FRED CSVs. This will only run if they actually exist.
else:
    macro_source = 'individual FRED CSVs'
    fred_files = {
        'DTB3': 'DTB3.csv',
        'DGS10': 'DGS10.csv',
```

```
        'VIX': 'VIXCLS.csv',
        'CPIAUCSL': 'CPIAUCSL.csv',
        'FEDFUNDS': 'FEDFUNDS.csv',
        'USREC': 'USREC.csv',
    }
    for col, fname in fred_files.items():
        path = find_csv(fname)
        if path is not None:
            file_col = 'VIXCLS' if fname == 'VIXCLS.csv' else col
            raw_macro[col] = _load_fred_from_file(path, file_col)

required = ['DTB3', 'VIX']
missing_required = [c for c in required if c not in raw_macro]
if missing_required:
    raise FileNotFoundError(
        'Missing required macro series: ' + ', '.join(missing_required) +
        '. Put macro_data.csv in the same project folder, or make sure returns_clean.csv contains DTB3 and VIX.'
    )

print(f'Macro source: {macro_source}')
print(f'Series loaded: {list(raw_macro.keys())}')
for name, s in raw_macro.items():
    s2 = s.dropna()
    if len(s2) == 0:
        print(f'  {name:<10} rows={len(s):>5}  all values missing')
    else:
        print(f'  {name:<10} rows={len(s):>5}  range={s2.index.min().date()} to {s2.index.max().date()}')
```

```
Macro source: macro_data.csv at /content/drive/MyDrive/5261 Final Project/macro_data.csv
Series loaded: ['DTB3', 'FEDFUNDS', 'DGS10', 'CPIAUCSL', 'VIX']
  DTB3       rows= 1748  range=2018-01-02 to 2024-12-31
  FEDFUNDS   rows= 1748  range=2018-01-02 to 2024-12-31
  DGS10      rows= 1748  range=2018-01-02 to 2024-12-31
  CPIAUCSL   rows= 1748  range=2018-01-02 to 2024-12-31
  VIX        rows= 1748  range=2018-01-02 to 2024-12-31
```

4. Align macro to the trading calendar

Rate and level series (DTB3, FEDFUNDS, DGS10, DFF, VIX) may have gaps on bank holidays when FRED doesn't publish but the stock market trades (Columbus Day, Veterans Day). Reindex to the master calendar, forward-fill short gaps, then linearly interpolate any remaining internal NaNs.

Monthly step-function series (CPIAUCSL, USREC) publish on the 1st of each month, which is often a weekend or holiday. Expand to a full calendar grid first so the first observation isn't lost, then forward-fill only (no interpolation, because CPI is discrete-release, not continuous).

```
master_index = prices.index
aligned = {}

for name in ['DTB3', 'FEDFUNDS', 'DGS10', 'DFF', 'VIX']:
    if name in raw_macro:
        aligned[name] = raw_macro[name].reindex(master_index).ffill().interpolate(method='linear')

for name in ['CPIAUCSL', 'USREC']:
    if name in raw_macro:
        s = raw_macro[name]
        start = min(s.index.min(), master_index.min())
        full_cal = pd.date_range(start, master_index.max(), freq='D')
        filled = s.reindex(full_cal).ffill().reindex(master_index)
        if name == 'USREC':
            filled = filled.astype('Int64')
        aligned[name] = filled

macro = pd.DataFrame(aligned)
print(f'Aligned macro panel: {macro.shape}')
print(f'NaNs per column: {macro.isna().sum().to_dict()}')
macro.head()
```

```
Aligned macro panel: (1760, 5)
NaNs per column: {'DTB3': 0, 'FEDFUNDS': 0, 'DGS10': 0, 'VIX': 0, 'CPIAUCSL': 0}
```

date	DTB3	FEDFUNDS	DGS10	VIX	CPIAUCSL
2018-01-02	1.42	1.42	2.46	9.77	248.859
2018-01-03	1.39	1.42	2.44	9.15	248.859
2018-01-04	1.39	1.42	2.46	9.22	248.859
2018-01-05	1.37	1.42	2.47	9.22	248.859
2018-01-08	1.43	1.42	2.49	9.52	248.859

Next steps: Generate code with `macro` New interactive sheet

5. Compute returns

- **Log returns:** $r_t = \ln(P_t/P_{t-1})$. Time-additive, which makes weekly aggregation clean.
- **Daily risk-free rate:** $r_f = \text{DTB3}/100/252$. DTB3 is annualized percent.
- **Excess returns:** $r_{i,t}^{\text{excess}} = r_{i,t} - r_{f,t}$. The dependent variable for the CAPM regression in the rolling beta section.

```
log_prices = np.log(prices)
returns = log_prices.diff()
returns.columns = [f'ret_{c}' for c in returns.columns]

rf_daily = (macro['DTB3'] / 100.0 / 252.0).rename('rf')

excess = returns.sub(rf_daily, axis=0)
excess.columns = [c.replace('ret_', 'excess_') for c in returns.columns]

returns_block = pd.concat([returns, rf_daily, excess], axis=1)
print(f'Returns block: {returns_block.shape}')
returns_block.head()
```

```
Returns block: (1760, 17)
```

date	ret_XLK	ret_XLE	ret_XLF	ret_XLV	ret_XLP	ret_XLU	ret_XLY	ret_SPY	rf	excess_XLK	excess_XLE	excess_
2018-01-02	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.000056	NaN	NaN	
2018-01-03	0.008307	0.014865	0.005358	0.009523	-0.000354	-0.007887	0.004581	0.006305	0.000055	0.008252	0.014810	0.00
2018-01-04	0.005042	0.006018	0.009220	0.001421	0.002827	-0.008339	0.003273	0.004206	0.000055	0.004987	0.005963	0.00
2018-01-05	0.010462	-0.000400	0.002820	0.008482	0.004401	-0.000390	0.007891	0.006642	0.000054	0.010407	-0.000455	0.00
2018-01-08	0.003764	0.005984	-0.001409	-0.003643	0.002456	0.009308	0.001178	0.001827	0.000057	0.003707	0.005928	-0.00

Next steps: Generate code with `returns_block` New interactive sheet

6. Regime and VIX labels

Load event windows from the macroeconomic `events.csv` (names normalized to lowercase and mapped to our internal labels; `Bank_crisis` → `svb_crisis`). If the file is missing, fall back to the hard-coded defaults above.

Overlaps are resolved by list order: later entries overwrite earlier ones. We sort by `REGIME_PRIORITY` so higher-priority (more specific) events come last.

```
events_path = DATA_DIR / 'events.csv'
if events_path.exists():
    df_ev = pd.read_csv(events_path)
    windows = []
    for _, row in df_ev.iterrows():
```

```
        raw_name = str(row['event_name']).strip().lower()
        label = EVENT_NAME_MAP.get(raw_name, raw_name)
        start = pd.Timestamp(str(row['start_date']).strip()).strftime('%Y-%m-%d')
        end   = pd.Timestamp(str(row['end_date']).strip()).strftime('%Y-%m-%d')
        windows.append((label, start, end))
    events_source = 'the macroeconomic data module events.csv'
else:
    windows = list(DEFAULT_EVENT_WINDOWS)
    events_source = 'default hard-coded windows'

windows.sort(key=lambda w: REGIME_PRIORITY.get(w[0], 0))
print(f'Event source: {events_source}')
for w in windows:
    print(f'  {w[0]:<18} {w[1]} to {w[2]}')

regime = pd.Series('normal', index=master_index, name='regime', dtype='object')
for label, start, end in windows:
    mask = (master_index >= pd.Timestamp(start)) & (master_index <= pd.Timestamp(end))
    regime.loc[mask] = label

vix_regime = pd.cut(macro['VIX'],
                    bins=[-np.inf, 15, 25, 35, np.inf],
                    labels=['low', 'medium', 'high', 'extreme']).rename('vix_regime')

print('\nRegime distribution:')
print(regime.value_counts())
print('\nVIX regime distribution:')
print(vix_regime.value_counts())
```

```
Event source: the macroeconomic data module events.csv
  rate_hike          2022-03-16 to 2023-07-26
  bear_2022          2022-01-03 to 2022-10-12
  ai_rally           2023-10-01 to 2024-12-31
  covid_recovery     2020-03-24 to 2020-12-31
  svb_crisis         2023-03-08 to 2023-05-01
  covid_crash        2020-02-19 to 2020-03-23

Regime distribution:
regime
normal            833
ai_rally          314
covid_recovery    197
bear_2022         196
rate_hike         158
svb_crisis         38
covid_crash        24
Name: count, dtype: int64

VIX regime distribution:
vix_regime
medium     920
low        512
high       272
extreme     56
Name: count, dtype: int64
```

7. Assemble the daily panel

```
daily = pd.concat([prices, returns_block, macro], axis=1)
daily['regime']     = regime
daily['vix_regime'] = vix_regime

col_order = (
    PRICE_COLS
    + [f'ret_{c}' for c in PRICE_COLS]
    + ['rf']
    + [f'excess_{c}' for c in PRICE_COLS]
    + ['VIX', 'DTB3', 'FEDFUNDS', 'DFF', 'DGS10', 'CPIAUCSL', 'USREC',
       'regime', 'vix_regime']
)
daily = daily[[c for c in col_order if c in daily.columns]]

print(f'Daily panel: {daily.shape}')
daily.head()
```

```
Daily panel: (1760, 32)
```

date	XLK	XLE	XLF	XLV	XLP	XLU	XLY	SPY	ret_XLK	ret_XLE	...	excess_XLU
2018-01-02	29.872919	25.747259	23.884136	72.723328	45.314568	20.132580	46.275238	236.562164	NaN	NaN	...	NaN
2018-01-03	30.122099	26.132856	24.012455	73.419174	45.298542	19.974422	46.487701	238.058426	0.008307	0.014865	...	-0.007942
2018-01-04	30.274368	26.290609	24.234875	73.523560	45.426765	19.808544	46.640114	239.061798	0.005042	0.006018	...	-0.008394
2018-01-05	30.592751	26.280090	24.303310	74.149803	45.627140	19.800827	47.009617	240.654953	0.010462	-0.000400	...	-0.000444
2018-01-08	30.708111	26.437834	24.269094	73.880180	45.739338	19.985998	47.065041	241.095062	0.003764	0.005984	...	0.009251

5 rows × 32 columns

8. Weekly aggregation

Each week ends on Friday. Log returns sum across the week. Everything else takes the last value of the week.

```
ret_cols    = [c for c in daily.columns if c.startswith('ret_')]
excess_cols = [c for c in daily.columns if c.startswith('excess_')]
last_cols   = [c for c in ['VIX', 'DTB3', 'FEDFUNDS', 'DFF', 'DGS10', 'rf',
                           'CPIAUCSL', 'USREC', 'regime', 'vix_regime'] + PRICE_COLS
               if c in daily.columns]

weekly_sum  = daily[ret_cols + excess_cols].resample('W-FRI').sum(min_count=1)
weekly_last = daily[last_cols].resample('W-FRI').last()
weekly = pd.concat([weekly_last, weekly_sum], axis=1).dropna(subset=[MARKET])

print(f'Weekly panel: {weekly.shape}')
weekly.head()
```

```
Weekly panel: (366, 32)
```

date	VIX	DTB3	FEDFUNDS	DGS10	rf	CPIAUCSL	regime	vix_regime	XLK	XLE	...	ret_XLY	ret_SPY	exc
2018-01-05	9.22	1.37	1.42	2.47	0.000054	248.859	normal	low	30.592751	26.280090	...	0.015745	0.017153	
2018-01-12	10.16	1.41	1.42	2.55	0.000056	248.859	normal	low	30.828068	27.138908	...	0.031432	0.016324	
2018-01-19	11.27	1.42	1.42	2.64	0.000056	248.859	normal	low	31.284895	26.774353	...	0.005412	0.008920	
2018-01-26	11.08	1.39	1.42	2.66	0.000055	248.859	normal	low	31.926291	27.170464	...	0.031690	0.021765	
2018-02-02	17.31	1.46	1.42	2.84	0.000058	249.529	normal	medium	30.675810	25.400221	...	-0.031784	-0.039612	

5 rows × 32 columns

9. Validation

```
# Check 1: no NaN in returns after 2018-01-05.
post = daily.loc['2018-01-05':, ret_cols]
n_nan = int(post.isna().sum().sum())
print(f'Check 1 - NaN in returns after 2018-01-05: {n_nan}  ({"PASS" if n_nan == 0 else "FAIL"})')

# Check 2: row count consistency.
ret_counts   = daily[ret_cols].count().unique()
price_counts = daily[PRICE_COLS].count().unique()
print(f'Check 2 - Return col row counts: {ret_counts[0]}  ({"PASS" if len(ret_counts) == 1 else "FAIL"})')
print(f'          Price col row counts:  {price_counts[0]}  ({"PASS" if len(price_counts) == 1 else "FAIL"})')
```

```
# Check 3: macro NaN count after warm-up.
macro_cols = [c for c in ['VIX','DTB3','FEDFUNDS','DFF','DGS10','CPIAUCSL','USREC'] if c in daily.columns]
n_macro_nan = int(daily.loc['2018-02-01':, macro_cols].isna().sum().sum())
print(f'Check 3 - NaN in macro after 2018-02-01: {n_macro_nan}  ({"PASS" if n_macro_nan == 0 else "FAIL"})')

# Sanity.
spy = daily['ret_SPY'].dropna()
print(f'\nSPY annualized vol: {spy.std() * np.sqrt(252):.2%}  (expect 15-25%)')

# Full-sample CAPM beta spot check.
for sec in ['XLK', 'XLP']:
    d2 = daily[[f'excess_{sec}', 'excess_SPY']].dropna()
    b = np.cov(d2[f'excess_{sec}'], d2['excess_SPY'])[0,1] / np.var(d2['excess_SPY'], ddof=1)
    print(f'Full-sample {sec} beta: {b:.3f}')

# Arithmetic identities.
assert np.allclose(daily['DTB3'] / 100 / 252, daily['rf']), 'rf formula mismatch'
assert np.allclose((daily['ret_XLK'] - daily['rf']).dropna(),
                   daily['excess_XLK'].dropna()), 'excess formula mismatch'
print('\nArithmetic identities: PASS')
```

```
Check 1 - NaN in returns after 2018-01-05: 0  (PASS)
Check 2 - Return col row counts: 1759  (PASS)
          Price col row counts:  1760  (PASS)
Check 3 - NaN in macro after 2018-02-01: 0  (PASS)

SPY annualized vol: 19.54%  (expect 15-25%)
Full-sample XLK beta: 1.251
Full-sample XLP beta: 0.600

Arithmetic identities: PASS
```

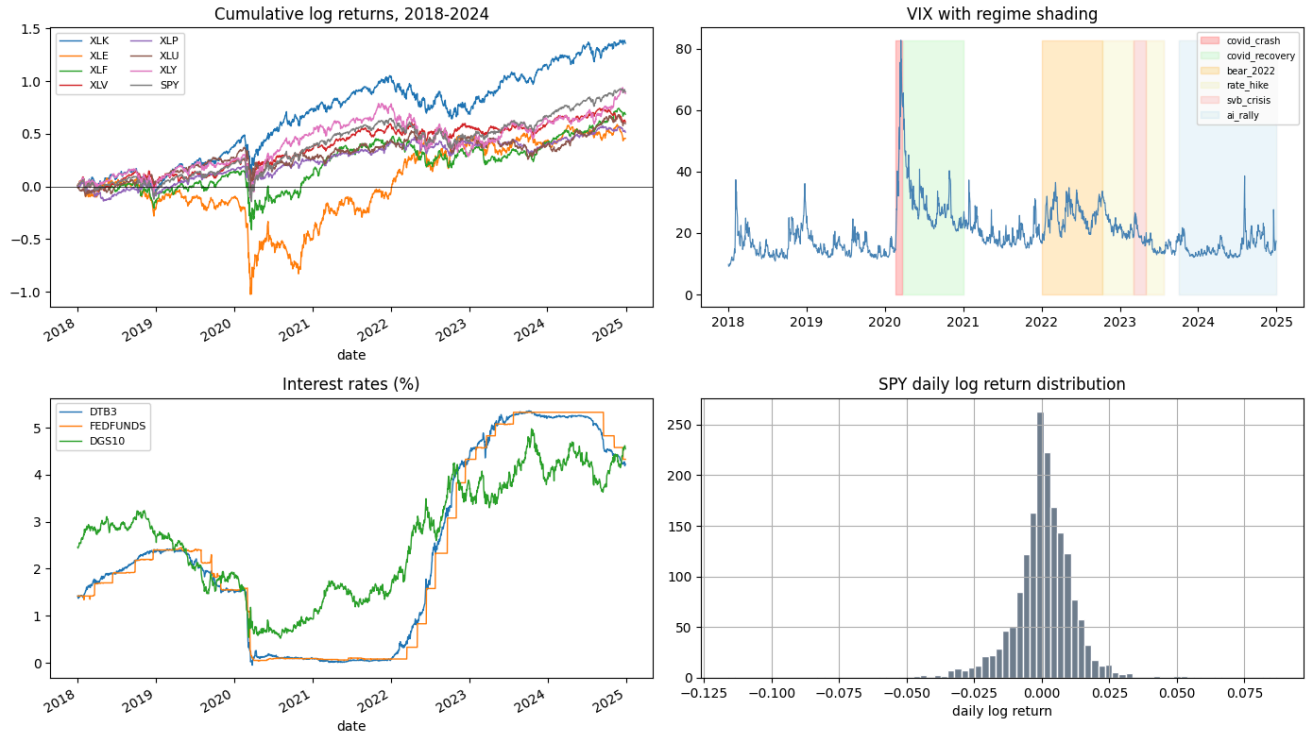
10. Write outputs

```
# Save outputs both in output/ and in the project root for compatibility with the rest of the notebook.
daily.to_csv(OUT_DIR / 'returns_clean.csv',         index_label='date')
weekly.to_csv(OUT_DIR / 'returns_clean_weekly.csv', index_label='date')
daily.to_csv(DATA_DIR / 'returns_clean.csv',         index_label='date')
weekly.to_csv(DATA_DIR / 'returns_clean_weekly.csv', index_label='date')

print(f'Wrote: {OUT_DIR / "returns_clean.csv"}')
print(f'Wrote: {OUT_DIR / "returns_clean_weekly.csv"}')
print(f'Wrote: {DATA_DIR / "returns_clean.csv"}')
print(f'Wrote: {DATA_DIR / "returns_clean_weekly.csv"}')
```

```
Wrote: /content/drive/MyDrive/5261 Final Project/output/returns_clean.csv
Wrote: /content/drive/MyDrive/5261 Final Project/output/returns_clean_weekly.csv
Wrote: /content/drive/MyDrive/5261 Final Project/returns_clean.csv
Wrote: /content/drive/MyDrive/5261 Final Project/returns_clean_weekly.csv
```

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

cum = daily[ret_cols].cumsum()
cum.columns = [c.replace('ret_', '') for c in cum.columns]
cum.plot(ax=axes[0, 0], linewidth=1)
axes[0, 0].set_title('Cumulative log returns, 2018-2024')
axes[0, 0].axhline(0, color='k', linewidth=0.5)
axes[0, 0].legend(loc='upper left', fontsize=8, ncol=2)

axes[0, 1].plot(daily.index, daily['VIX'], color='steelblue', linewidth=0.8)
regime_colors = {'covid_crash': 'red', 'covid_recovery': 'lightgreen',
                 'bear_2022': 'orange', 'rate_hike': 'khaki',
                 'svb_crisis': 'salmon', 'ai_rally': 'lightblue'}
for label, color in regime_colors.items():
    mask = daily['regime'] == label
    if mask.any():
        axes[0, 1].fill_between(daily.index, 0, daily['VIX'].max(),
                                where=mask, alpha=0.2, color=color, label=label)
axes[0, 1].set_title('VIX with regime shading')
axes[0, 1].legend(fontsize=7, loc='upper right')

rate_cols_plot = [c for c in ['DTB3', 'FEDFUNDS', 'DGS10'] if c in daily.columns]
```

```
daily[rate_cols_plot].plot(ax=axes[1, 0], linewidth=1)
axes[1, 0].set_title('Interest rates (%)')
axes[1, 0].legend(fontsize=8)

daily['ret_SPY'].dropna().hist(ax=axes[1, 1], bins=80, color='slategray', edgecolor='white')
axes[1, 1].set_title('SPY daily log return distribution')
axes[1, 1].set_xlabel('daily log return')

plt.tight_layout()
plt.show()
```



3. Descriptive Statistics and Preliminary Analysis

Setup

```
import numpy as np
import pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt
import seaborn as sns

# DATA_DIR, OUT_DIR, and FIGURE_OUT_DIR are defined in the setup cell.
OUT_DIR.mkdir(parents=True, exist_ok=True)
FIGURE_OUT_DIR.mkdir(parents=True, exist_ok=True)

SECTORS = ['XLK', 'XLE', 'XLF', 'XLV', 'XLP', 'XLU', 'XLY']
MARKET = 'SPY'
TRADING_DAYS = 252

sns.set_theme(style="whitegrid")
```

1. Load Data

```
daily_path = find_csv('returns_clean.csv')
if daily_path is None:
    # If the data-cleaning section just produced the file, use the output path.
    daily_path = OUT_DIR / 'returns_clean.csv'

daily = pd.read_csv(daily_path, parse_dates=["date"])
daily = daily.sort_values("date").set_index("date")

print("Loaded:", daily_path)
print("Shape:", daily.shape)
print("Date range:", daily.index.min().date(), "to", daily.index.max().date())

ret_cols = [f"ret_{s}" for s in SECTORS + [MARKET]]
excess_cols = [f"excess_{s}" for s in SECTORS + [MARKET]]

print("Return columns:", ret_cols)
print("Excess return columns:", excess_cols)
```

```
Loaded: /content/drive/MyDrive/5261 Final Project/returns_clean.csv
Shape: (1760, 32)
Date range: 2018-01-02 to 2024-12-30
Return columns: ['ret_XLK', 'ret_XLE', 'ret_XLF', 'ret_XLV', 'ret_XLP', 'ret_XLU', 'ret_XLY', 'ret_SPY']
Excess return columns: ['excess_XLK', 'excess_XLE', 'excess_XLF', 'excess_XLV', 'excess_XLP', 'excess_XLU', 'excess_X
```

2. Function

```
def annualized_return(log_ret):
    return log_ret.mean() * TRADING_DAYS

def annualized_volatility(log_ret):
    return log_ret.std(ddof=1) * np.sqrt(TRADING_DAYS)

def sharpe_ratio(excess_ret):
    return (excess_ret.mean() / excess_ret.std(ddof=1)) * np.sqrt(TRADING_DAYS)

def max_drawdown_from_log_returns(log_ret):
    cum_index = np.exp(log_ret.cumsum())
    running_max = cum_index.cummax()
    drawdown = cum_index / running_max - 1.0
    return drawdown.min()

def static_beta(asset_ret, market_ret):
    return np.cov(asset_ret, market_ret, ddof=1)[0, 1] / np.var(market_ret, ddof=1)
```

3. Annual Return, Volatility, Sharpe, and Static Beta Table

```
rows = []

for s in SECTORS:
    r = daily[f"ret_{s}"].dropna()
    ex = daily[f"excess_{s}"].dropna()
    rm = daily["ret_SPY"].dropna()

    common_idx = r.index.intersection(rm.index)
    beta_val = static_beta(r.loc[common_idx], rm.loc[common_idx])

    rows.append({
        "Sector": s,
        "Annualized Return": annualized_return(r),
        "Annualized Volatility": annualized_volatility(r),
        "Sharpe Ratio": sharpe_ratio(ex),
        "Max Drawdown": max_drawdown_from_log_returns(r),
        "Skewness": r.skew(),
        "Excess Kurtosis": r.kurt(),
        "Static Beta": beta_val
    })
```

```
stats_table = pd.DataFrame(rows).set_index("Sector")
stats_table = stats_table.round(4)

print(stats_table)

stats_table.to_csv(OUT_DIR / "stats_table.csv")
print("Saved:", OUT_DIR / "stats_table.csv")
```

```
        Annualized Return  Annualized Volatility  Sharpe Ratio  Max Drawdown  \
Sector
XLK                0.1948                 0.2618        0.6543       -0.3356
XLE                0.0654                 0.3293        0.1273       -0.6681
XLF                0.0981                 0.2427        0.3075       -0.4286
XLV                0.0879                 0.1761        0.3656       -0.2840
XLP                0.0738                 0.1588        0.3170       -0.2451
XLU                0.0855                 0.2098        0.2957       -0.3607
XLY                0.1264                 0.2391        0.4306       -0.3967

        Skewness  Excess Kurtosis  Static Beta
Sector
XLK      -0.4380           7.7983       1.2506
XLE      -0.9114          15.2633       1.0435
XLF      -0.5872          14.9296       1.0577
XLV      -0.4432          10.7165       0.7506
XLP      -0.5111          15.8658       0.5999
XLU      -0.2170          15.5082       0.6420
XLY      -0.7568           7.6350       1.1040
Saved: /content/drive/MyDrive/5261 Final Project/output/stats_table.csv
```

Interpretation of Summary Statistics

The summary statistics reveal substantial differences in risk and return characteristics across sectors over the sample period. Among all sectors, XLK (Technology) delivered the highest annualized return at 19.48%, while also maintaining the highest Sharpe ratio (0.6543), indicating the strongest risk-adjusted performance. This suggests that technology was the best-performing sector overall, likely benefiting from strong growth momentum during the later part of the sample period.

In contrast, XLE (Energy) exhibited the highest annualized volatility at 32.93% and the most severe maximum drawdown at -66.81%, indicating that it was the most volatile and vulnerable sector during market stress. Although its annualized return was positive, its low Sharpe ratio (0.1273) suggests that these returns were achieved with relatively poor risk-adjusted efficiency.

The defensive sectors, particularly XLP (Consumer Staples) and XLV (Healthcare), showed lower annualized volatility and smaller drawdowns compared with cyclical sectors. For example, XLP had the lowest static beta (0.5999) and one of the smallest maximum drawdowns (-24.51%), confirming its defensive nature. Similarly, XLV displayed relatively low volatility (17.61%) and a moderate Sharpe ratio (0.3656), suggesting a more stable return profile.

The static beta estimates further support the distinction between cyclical and defensive sectors. XLK (1.2506) and XLY (1.1040) had the highest betas, indicating greater sensitivity to overall market movements, while XLP (0.5999) and XLU (0.6420) had much lower betas, consistent with their role as lower-risk sectors. This pattern suggests that cyclical sectors tend to amplify market movements, whereas defensive sectors provide relatively more stability.

Finally, the negative skewness across all sectors indicates that returns were generally characterized by larger downside moves than upside surprises, while the high excess kurtosis values suggest fat-tailed distributions and the presence of extreme return events. This implies that sector returns were not normally distributed and that tail risk was an important feature of the sample period.

Overall, these descriptive results provide a clear baseline for the later rolling beta analysis: cyclical sectors such as XLK, XLY, and XLE tend to offer higher return potential but also higher risk and market sensitivity, while defensive sectors such as XLP and XLV exhibit more stable behavior and lower systematic risk.

4. Correlation Matrix

```
sector_ret = daily[[f"ret_{s}" for s in SECTORS]]
corr_matrix = sector_ret.corr().round(4)

print(corr_matrix)

corr_matrix.to_csv(OUT_DIR / "correlation_matrix.csv")
print("Saved:", OUT_DIR / "correlation_matrix.csv")
```

```
         ret_XLK  ret_XLE  ret_XLF  ret_XLV  ret_XLP  ret_XLU  ret_XLY
ret_XLK   1.0000   0.4514   0.6787   0.7142   0.5938   0.4441   0.8505
ret_XLE   0.4514   1.0000   0.7105   0.4879   0.4271   0.3832   0.4875
```

```
ret_XLF   0.6787   0.7105   1.0000   0.7061   0.6485   0.5494   0.7301
ret_XLV   0.7142   0.4879   0.7061   1.0000   0.7531   0.6265   0.6625
ret_XLP   0.5938   0.4271   0.6485   0.7531   1.0000   0.7312   0.5949
ret_XLU   0.4441   0.3832   0.5494   0.6265   0.7312   1.0000   0.4606
ret_XLY   0.8505   0.4875   0.7301   0.6625   0.5949   0.4606   1.0000
Saved: /content/drive/MyDrive/5261 Final Project/output/correlation_matrix.csv
```

Interpretation of Correlation Matrix

The correlation matrix complements the summary statistics by showing how sector returns move together over time. All sectors are positively correlated, indicating the presence of common market-wide influences. The strongest correlation is observed between XLK and XLY (0.8505), suggesting that technology and consumer discretionary sectors are highly synchronized and likely respond similarly to growth expectations and market sentiment. High correlations among XLF, XLK, and XLY also indicate that cyclical sectors tend to move together more closely. By contrast, XLU generally has lower correlations with most other sectors, implying relatively better diversification potential. Overall, the correlation results are consistent with the summary statistics: cyclical sectors not only have higher volatility and beta, but also exhibit stronger co-movement, while defensive sectors are more stable yet still affected by common macroeconomic shocks.

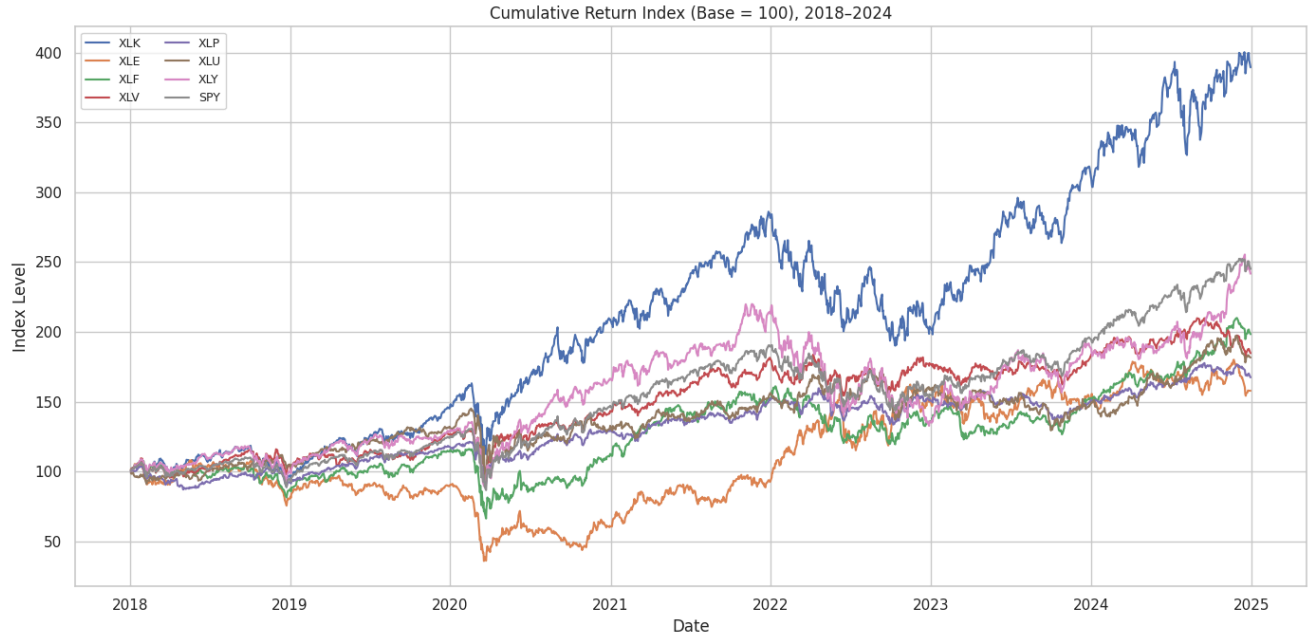
5. Static Beta Table

```
beta_rows = []
market_ret = daily["ret_SPY"]

for s in SECTORS:
    asset_ret = daily[f"ret_{s}"]
    common_idx = asset_ret.dropna().index.intersection(market_ret.dropna().index)

    beta_rows.append({
        "Sector": s,
        "Static Beta": static_beta(asset_ret.loc[common_idx], market_ret.loc[common_idx])
    })

beta_table = pd.DataFrame(beta_rows).set_index("Sector").round(4)

print(beta_table)

beta_table.to_csv(OUT_DIR / "static_beta_table.csv")
print("Saved:", OUT_DIR / "static_beta_table.csv")
```

```
        Static Beta
Sector
XLK          1.2506
XLE          1.0435
XLF          1.0577
XLV          0.7506
XLP          0.5999
XLU          0.6420
XLY          1.1040
Saved: /content/drive/MyDrive/5261 Final Project/output/static_beta_table.csv
```

6. Cumulative Return Chart

```
cum_log = daily[[f"ret_{s}" for s in SECTORS + [MARKET]]].cumsum()
cum_index = np.exp(cum_log) * 100
cum_index.columns = [c.replace("ret_", "") for c in cum_index.columns]

plt.figure(figsize=(14, 7))
for col in cum_index.columns:
    plt.plot(cum_index.index, cum_index[col], linewidth=1.5, label=col)

plt.title("Cumulative Return Index (Base = 100), 2018–2024")
plt.xlabel("Date")
plt.ylabel("Index Level")
plt.legend(ncol=2, fontsize=9)
plt.tight_layout()
plt.savefig(OUT_DIR / "cumulative_return_index.png", dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", OUT_DIR / "cumulative_return_index.png")
```



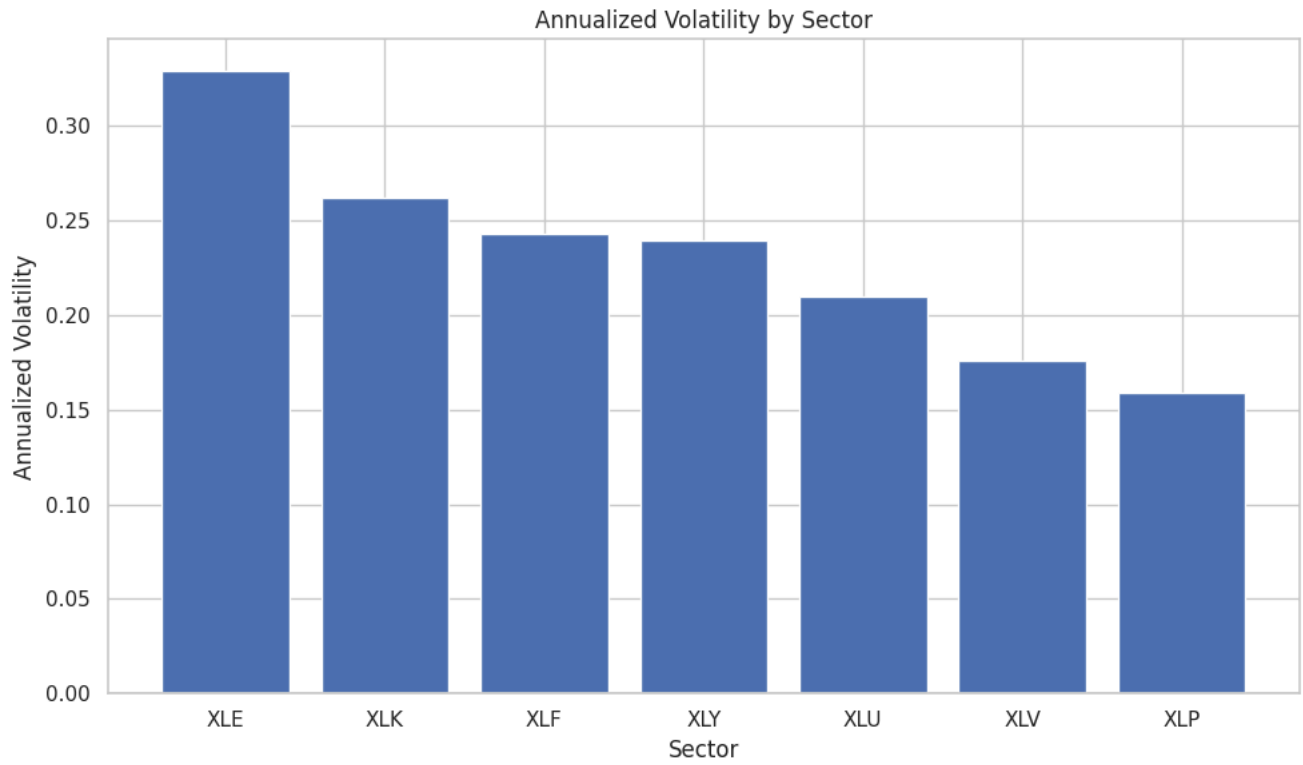

```
Saved: /content/drive/MyDrive/5261 Final Project/output/cumulative_return_index.png
```

Figure 1. Cumulative Return Index

The cumulative return index shows strong divergence in sector performance over time. XLK clearly delivered the highest long-run growth and outperformed SPY by a wide margin, indicating that technology was the strongest-performing sector during the sample period. In contrast, XLE experienced the largest drop during the 2020 shock and remained the weakest performer for much of the period. Defensive sectors such as XLP, XLV, and XLU followed smoother but less aggressive growth paths.

7. Annualized Volatility Bar Chart

```
vol_table = stats_table["Annualized Volatility"].sort_values(ascending=False)

plt.figure(figsize=(10, 6))
plt.bar(vol_table.index, vol_table.values)
plt.title("Annualized Volatility by Sector")
plt.xlabel("Sector")
plt.ylabel("Annualized Volatility")
plt.tight_layout()
plt.savefig(OUT_DIR / "annualized_volatility_bar.png", dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", OUT_DIR / "annualized_volatility_bar.png")
```




```
Saved: /content/drive/MyDrive/5261 Final Project/output/annualized_volatility_bar.png
```

Figure 2. Annualized Volatility by Sector

The volatility chart shows that risk differs substantially across sectors. XLE had the highest annualized volatility, followed by XLK, XLF, and XLY, suggesting that cyclical sectors faced greater return fluctuations. Meanwhile, XLP and XLV had the lowest volatility, supporting their role as more defensive sectors. This pattern indicates a clear trade-off between return potential and stability.

8. Correlation Heatmap

```
plt.figure(figsize=(8, 6))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    center=0,
    fmt=".2f",
    linewidths=0.5,
    square=True
)

plt.title("Correlation Heatmap of Sector Returns")
plt.tight_layout()
plt.savefig(OUT_DIR / "correlation_heatmap.png", dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", OUT_DIR / "correlation_heatmap.png")
```

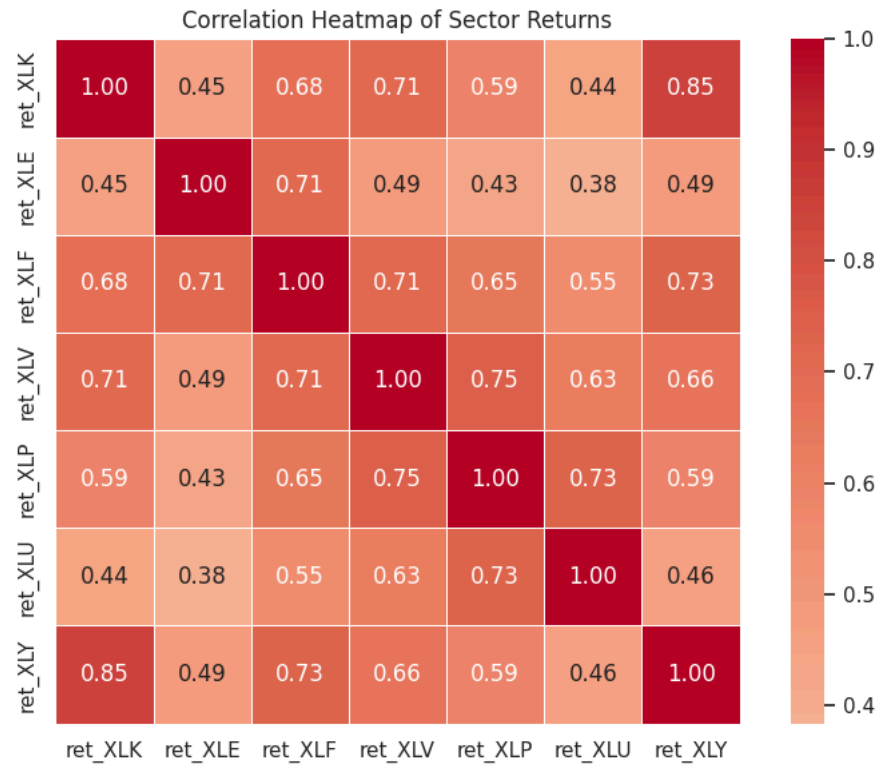
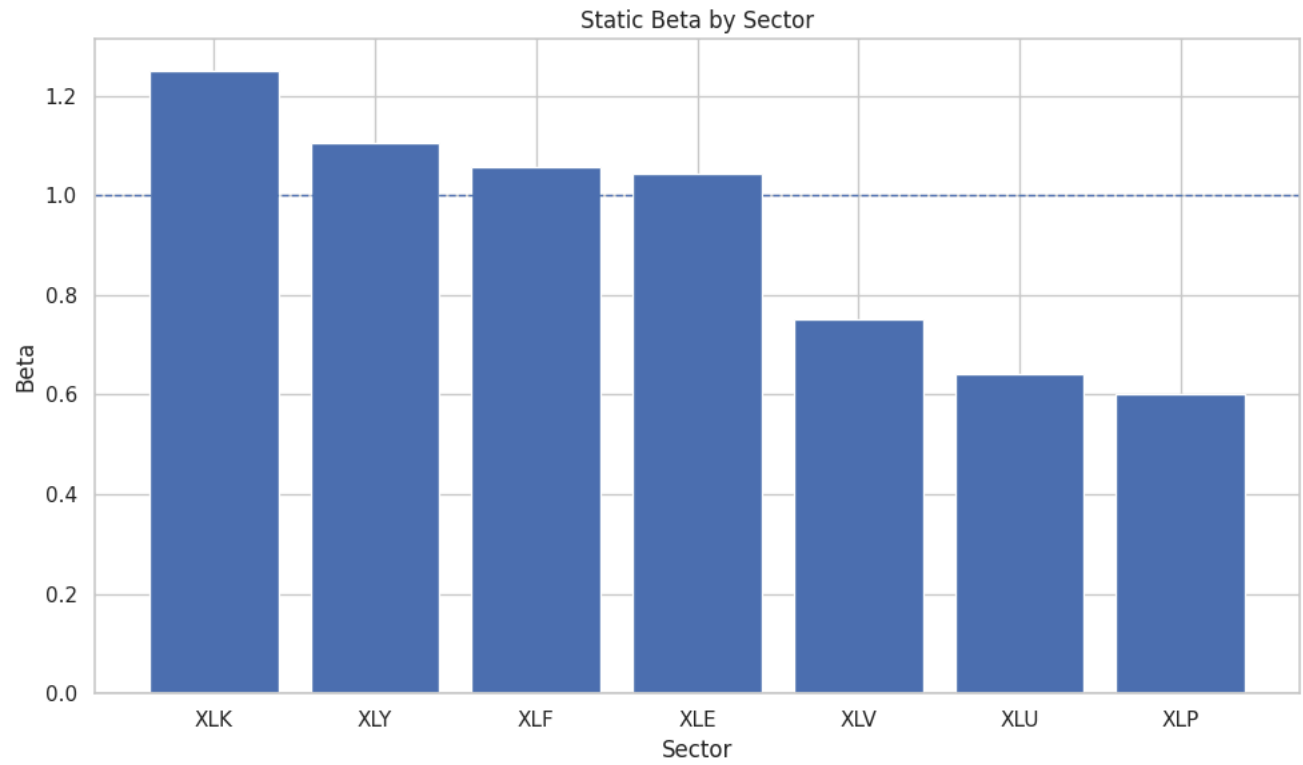


Figure 3. Correlation Heatmap

The correlation heatmap shows that all sectors are positively correlated, meaning that they are all affected by common market-wide conditions. However, some sectors move much more closely together than others. XLK and XLY have the strongest correlation, while XLU generally shows weaker relationships with the rest of the sectors. This suggests that diversification benefits exist, but they are limited because most sectors still share substantial co-movement.

9. Static Beta Bar Chart

```
beta_plot = beta_table["Static Beta"].sort_values(ascending=False)

plt.figure(figsize=(10, 6))
plt.bar(beta_plot.index, beta_plot.values)
plt.axhline(1.0, linestyle="--", linewidth=1)
plt.title("Static Beta by Sector")
plt.xlabel("Sector")
plt.ylabel("Beta")
plt.tight_layout()
plt.savefig(OUT_DIR / "static_beta_bar.png", dpi=300, bbox_inches="tight")
plt.show()

print("Saved:", OUT_DIR / "static_beta_bar.png")
```




```
Saved: /content/drive/MyDrive/5261 Final Project/output/static_beta_bar.png
```

Figure 4. Static Beta by Sector

The static beta chart shows that XLK has the highest beta, followed by XLY, XLF, and XLE, indicating that these sectors are more sensitive to overall market movements. By contrast, XLP, XLU, and XLV have lower beta values, confirming their more defensive nature. The beta ranking is consistent with the volatility and return results and further supports the distinction between cyclical and defensive sectors.

4. Rolling Beta Estimation

This section estimates rolling CAPM betas for seven sector ETFs using weekly excess returns from 2018 to 2024. The main output is `rolling_betas.csv`, which contains rolling beta, alpha, R-squared, and beta standard error for each sector. A 26-week robustness check is also produced for XLK and XLP.

Model:

$$r_{i,t} - r_{f,t} = lpha_i + eta_{i,t}(r_{m,t} - r_{f,t}) + arepsilon_{i,t}$$

Each rolling window runs OLS on the preceding 52 weeks. The estimated $eta_{i,t}$ is interpreted as the time-varying systematic risk exposure of sector i at time t.

Setup

```
# DATA_DIR and OUT_DIR are already defined in the setup cell.
# Do not remount or reset the path here, otherwise Colab may point to the wrong folder.
OUT_DIR.mkdir(parents=True, exist_ok=True)
print(f'DATA_DIR: {DATA_DIR}')
print(f'OUT_DIR : {OUT_DIR}')
```

```
DATA_DIR: /content/drive/MyDrive/5261 Final Project
OUT_DIR : /content/drive/MyDrive/5261 Final Project/output
```

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import warnings
```

```
warnings.filterwarnings('ignore')

SECTORS  = ['XLK', 'XLE', 'XLF', 'XLV', 'XLP', 'XLU', 'XLY']
MARKET   = 'SPY'
WINDOW_MAIN   = 52   # weeks — primary model
WINDOW_ROBUST = 26   # weeks — robustness check (XLK + XLP only)

# Colour palette for plots (one colour per sector + SPY)
SECTOR_COLORS = {
    'XLK': '#534AB7', 'XLE': '#854F0B', 'XLF': '#185FA5',
    'XLV': '#0F6E56', 'XLP': '#993C1D', 'XLU': '#3B6D11', 'XLY': '#888780'
}

# Event windows for shading plots
EVENT_WINDOWS = [
    ('covid_crash',    '2020-02-19', '2020-03-23', '#f7c1c1'),
    ('rate_hike',      '2022-03-16', '2023-07-26', '#fac775'),
    ('svb_crisis',     '2023-03-08', '2023-05-01', '#f0997b'),
    ('ai_rally',       '2023-10-01', '2024-12-31', '#9FE1CB'),
]

print('Setup complete.')
```

```
Setup complete.
```

2. Load Data

```
weekly_path = find_csv('returns_clean_weekly.csv')
if weekly_path is None:
    weekly_path = OUT_DIR / 'returns_clean_weekly.csv'

weekly = pd.read_csv(weekly_path, parse_dates=['date'], index_col='date')
weekly = weekly.sort_index()

print('Loaded:', weekly_path)
print('Shape:', weekly.shape)
print('Date range:', weekly.index.min().date(), 'to', weekly.index.max().date())
print('\nExcess return columns available:')
print([c for c in weekly.columns if c.startswith('excess_')])
```

```
Loaded: /content/drive/MyDrive/5261 Final Project/returns_clean_weekly.csv
Shape: (366, 32)
Date range: 2018-01-05 to 2025-01-03

Excess return columns available:
['excess_XLK', 'excess_XLE', 'excess_XLF', 'excess_XLV', 'excess_XLP', 'excess_XLU', 'excess_XLY', 'excess_SPY']
```

3. Rolling Beta Estimation — Main Model (52-week)

For each sector, we slide a 52-week window one week at a time and run OLS:

$$\text{excess}_{i,t} = \alpha + \beta \cdot \text{excess}_{SPY,t} + \varepsilon_t$$

We record four values per window: **beta**, **alpha**, **R²**, and **beta standard error** (a proxy for estimation uncertainty).

The output is a tidy long-format DataFrame: one row per (date, sector) pair.

```
def rolling_ols(df, sector, window):
    """
    Run rolling OLS for one sector.
    Returns a DataFrame with columns: [date, sector, beta, alpha, r2, beta_se, window].
    """
    y_col = f'excess_{sector}'
    x_col = f'excess_{MARKET}'

    y = df[y_col].values
    x = df[x_col].values
    dates = df.index

    records = []
    for end in range(window, len(df) + 1):
        start = end - window
        y_win = y[start:end]
```

```
            x_win = x[start:end]

            # Drop any NaN rows within the window
            mask = ~(np.isnan(y_win) | np.isnan(x_win))
            if mask.sum() < window * 0.8:   # require at least 80% non-NaN
                continue

            X = sm.add_constant(x_win[mask])
            model = sm.OLS(y_win[mask], X).fit()

            records.append({
                'date'    : dates[end - 1],          # label = last date of window
                'sector'  : sector,
                'beta'    : model.params[1],
                'alpha'   : model.params[0],
                'r2'      : model.rsquared,
                'beta_se' : model.bse[1],
                'window'  : window,
            })

        return pd.DataFrame(records)


    # Run for all 7 sectors, 52-week window
    results_52w = []
    for sector in SECTORS:
        df_s = rolling_ols(weekly, sector, WINDOW_MAIN)
        results_52w.append(df_s)
        print(f'{sector}: {len(df_s)} rolling windows estimated')

    rolling_betas = pd.concat(results_52w, ignore_index=True)
    print(f'\nTotal rows: {len(rolling_betas)}')
    rolling_betas.head(14)
```

```
XLK: 315 rolling windows estimated
XLE: 315 rolling windows estimated
XLF: 315 rolling windows estimated
XLV: 315 rolling windows estimated
XLP: 315 rolling windows estimated
XLU: 315 rolling windows estimated
XLY: 315 rolling windows estimated

Total rows: 2205
```

	date	sector	beta	alpha	r2	beta_se	window
0	2018-12-28	XLK	1.118500	0.000648	0.887339	0.056363	52
1	2019-01-04	XLK	1.104421	0.000170	0.878032	0.058213	52
2	2019-01-11	XLK	1.113151	0.000462	0.882908	0.057329	52
3	2019-01-18	XLK	1.108497	0.000297	0.884438	0.056666	52
4	2019-01-25	XLK	1.110805	0.000598	0.881265	0.057662	52
5	2019-02-01	XLK	1.111545	0.000395	0.876683	0.058957	52
6	2019-02-08	XLK	1.133298	0.000488	0.868195	0.062448	52
7	2019-02-15	XLK	1.119383	0.000271	0.860686	0.063690	52
8	2019-02-22	XLK	1.119032	0.000225	0.861478	0.063459	52
9	2019-03-01	XLK	1.130527	0.000035	0.865786	0.062949	52
10	2019-03-08	XLK	1.127677	0.000073	0.861861	0.063847	52
11	2019-03-15	XLK	1.145218	0.000262	0.864224	0.064195	52
12	2019-03-22	XLK	1.113322	0.000753	0.838210	0.069173	52
13	2019-03-29	XLK	1.115306	0.000763	0.837547	0.069465	52

Next steps: Generate code with `rolling_betas` New interactive sheet

4. Validation

Basic sanity checks before saving:

- Beta range should be broadly between −0.5 and 2.5 for sector ETFs

- R^2 should be positive and below 1
- No NaN in key columns

```
print('=== Validation ===')

# Check 1: NaN
nan_count = rolling_betas[['beta','alpha','r2','beta_se']].isna().sum().sum()
print(f'NaN in key columns: {nan_count}  ({"PASS" if nan_count == 0 else "FAIL"})')

# Check 2: Beta range
b_min, b_max = rolling_betas['beta'].min(), rolling_betas['beta'].max()
print(f'Beta range: [{b_min:.3f}, {b_max:.3f}]  ({"PASS" if -0.5 <= b_min and b_max <= 2.5 else "CHECK"})')

# Check 3: R² range
r2_min, r2_max = rolling_betas['r2'].min(), rolling_betas['r2'].max()
print(f'R² range:   [{r2_min:.3f}, {r2_max:.3f}]  ({"PASS" if 0 <= r2_min and r2_max <= 1 else "FAIL"})')

# Check 4: All sectors present
found = sorted(rolling_betas['sector'].unique().tolist())
print(f'Sectors found: {found}  ({"PASS" if found == sorted(SECTORS) else "FAIL"})')

# Summary stats by sector
print('\n=== Beta summary by sector ===')
rolling_betas.groupby('sector')['beta'].agg(['mean','std','min','max']).round(3)
```

```
=== Validation ===
NaN in key columns: 0  (PASS)
Beta range: [0.150, 1.828]  (PASS)
R² range:   [0.011, 0.949]  (PASS)
Sectors found: ['XLE', 'XLF', 'XLK', 'XLP', 'XLU', 'XLV', 'XLY']  (PASS)

=== Beta summary by sector ===
```

sector	mean	std	min	max
XLE	0.922	0.362	0.230	1.692
XLF	1.034	0.148	0.570	1.281
XLK	1.198	0.172	0.936	1.828
XLP	0.593	0.129	0.204	0.799
XLU	0.589	0.299	0.150	1.194
XLV	0.748	0.166	0.413	1.093
XLY	1.193	0.124	0.961	1.452

5. Save Main Output

```
out_path = OUT_DIR / 'rolling_betas.csv'
rolling_betas.to_csv(out_path, index=False)
print(f'Saved: {out_path}')
print(f'Rows: {len(rolling_betas)},  Columns: {list(rolling_betas.columns)}')
```

```
Saved: /content/drive/MyDrive/5261 Final Project/output/rolling_betas.csv
Rows: 2205,  Columns: ['date', 'sector', 'beta', 'alpha', 'r2', 'beta_se', 'window']
```

6. Robustness Check — 26-week Window (XLK and XLP only)

We re-estimate beta using a 26-week (half-year) window for two representative sectors:

- **XLK** (Technology) — highest beta, most cyclical
- **XLP** (Consumer Staples) — lowest beta, most defensive

The goal is to show that the core conclusion (cyclical vs. defensive separation) holds regardless of window choice. We do **not** run this for all 7 sectors to avoid chart overload.

```
results_26w = []
for sector in ['XLK', 'XLP']:
    df_s = rolling_ols(weekly, sector, WINDOW_ROBUST)
```

```
    results_26w.append(df_s)
    print(f'{sector} (26w): {len(df_s)} windows')

rolling_betas_26w = pd.concat(results_26w, ignore_index=True)

out_path_26w = OUT_DIR / 'rolling_betas_26w.csv'
rolling_betas_26w.to_csv(out_path_26w, index=False)
print(f'\nSaved: {out_path_26w}')
```

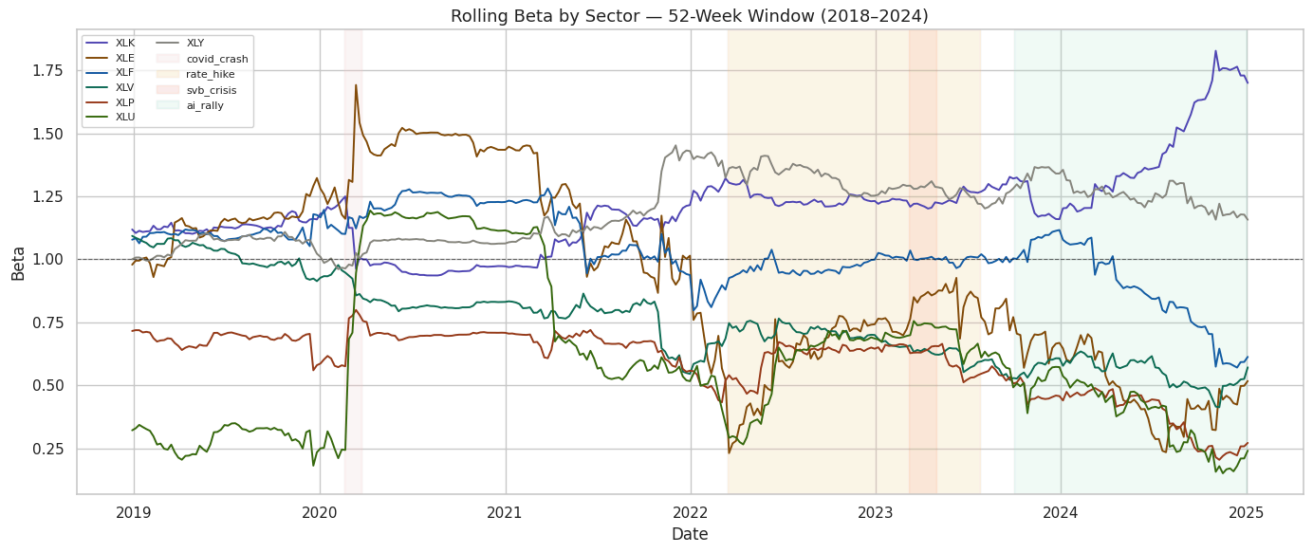
```
XLK (26w): 341 windows
XLP (26w): 341 windows

Saved: /content/drive/MyDrive/5261 Final Project/output/rolling_betas_26w.csv
```

7. Quick Preview Charts

These are diagnostic plots for rolling beta own verification. The polished versions for presentation will be produced by visualization.

7A. Rolling Beta — All Sectors (52-week)

```
def shade_events(ax):
    """Add semi-transparent event window shading to an axes."""
    for name, start, end, color in EVENT_WINDOWS:
        ax.axvspan(pd.Timestamp(start), pd.Timestamp(end),
                   alpha=0.15, color=color, label=name)

# Pivot to wide format for easy plotting
beta_wide = rolling_betas.pivot(index='date', columns='sector', values='beta')

fig, ax = plt.subplots(figsize=(14, 6))

for sector in SECTORS:
    ax.plot(beta_wide.index, beta_wide[sector],
            label=sector, color=SECTOR_COLORS[sector], linewidth=1.4)

shade_events(ax)
ax.axhline(1.0, color='black', linestyle='--', linewidth=0.8, alpha=0.5)
ax.set_title('Rolling Beta by Sector — 52-Week Window (2018–2024)', fontsize=13)
ax.set_xlabel('Date')
ax.set_ylabel('Beta')
ax.legend(loc='upper left', fontsize=8, ncol=2)
plt.tight_layout()
plt.savefig(OUT_DIR / 'preview_rolling_beta_52w.png', dpi=150)
plt.show()
```



Interpretation — Rolling Beta Line Chart

The 52-week rolling beta chart reveals several patterns consistent with our research hypotheses.

Sector stratification is stable across the full sample. XLK and XLY consistently sit above 1.0 throughout the period, while XLP and XLU remain below 1.0 in most regimes. This confirms the cyclical vs. defensive separation holds not just on average, but dynamically over time.

COVID crash (Feb–Mar 2020). XLE shows a sharp spike to above 1.6 before reverting quickly, reflecting the extreme demand shock to energy during lockdowns. XLK, by contrast, saw its beta briefly decline — consistent with technology being treated as a relative safe haven during the initial crash.

Rate hike cycle (Mar 2022–Jul 2023). XLU's beta rises from around 0.5 toward 1.0 during this period, supporting the hypothesis that interest rate risk dominated its defensive profile when rates increased aggressively. XLF shows elevated and volatile beta throughout, reflecting the dual pressure of tightening conditions and the SVB crisis in early 2023.

2024 AI rally (Oct 2023 onward). XLK's beta climbs steadily to 1.7–1.8, the highest of any sector across the entire sample. This reflects the growing concentration of S&P 500 returns in a small number of mega-cap tech names — as XLK became the dominant driver of SPY, its measured co-movement with the market naturally increased.

Late-period decline in XLP and XLU. Both defensive sectors show unusually low betas (approaching 0.2–0.3) toward 2024–2025, suggesting increasing decoupling from the broad market as growth and tech narratives dominated price action.

5. Rolling Beta Visualization

Rolling beta plots for sector ETFs

- This section visualizes the rolling CAPM beta estimates for sector ETFs.
- The goal is to examine how market exposure evolves over time and across different market conditions.

```
# DATA_DIR and OUT_DIR are defined in the setup cell.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

# DATA_DIR and OUT_DIR are defined in the setup cell.
FIGURE_OUT_DIR = OUT_DIR / "figures"
FIGURE_OUT_DIR.mkdir(exist_ok=True)
```

1. Load data

- Rolling beta results from rolling beta estimation
- Weekly returns and regime data

```
rolling_52 = pd.read_csv(
    OUT_DIR / "rolling_betas.csv",
    parse_dates=["date"]
)

rolling_26 = pd.read_csv(
    OUT_DIR / "rolling_betas_26w.csv",
    parse_dates=["date"]
)

weekly_path = find_csv('returns_clean_weekly.csv') or (OUT_DIR / 'returns_clean_weekly.csv')
weekly = pd.read_csv(
    weekly_path,
    parse_dates=["date"]
)

rolling_52 = rolling_52.sort_values(["sector", "date"]).reset_index(drop=True)
rolling_26 = rolling_26.sort_values(["sector", "date"]).reset_index(drop=True)
weekly = weekly.sort_values("date").reset_index(drop=True)
```

```
sector_order = sorted(rolling_52["sector"].unique())

print("rolling_52 shape:", rolling_52.shape)
print("rolling_26 shape:", rolling_26.shape)
print("weekly shape:", weekly.shape)
print("weekly loaded from:", weekly_path)
print("\nSectors:", sector_order)

print(
    "\n52-week rolling beta date range:",
    rolling_52["date"].min().date(),
    "to",
    rolling_52["date"].max().date()
)

print(
    "26-week rolling beta date range:",
    rolling_26["date"].min().date(),
    "to",
    rolling_26["date"].max().date()
)

print("\nMissing beta in rolling_52:", rolling_52["beta"].isna().sum())
print("Missing beta in rolling_26:", rolling_26["beta"].isna().sum())
```

```
rolling_52 shape: (2205, 7)
rolling_26 shape: (682, 7)
weekly shape: (366, 33)
weekly loaded from: /content/drive/MyDrive/5261 Final Project/returns_clean_weekly.csv

Sectors: ['XLE', 'XLF', 'XLK', 'XLP', 'XLU', 'XLV', 'XLY']

52-week rolling beta date range: 2018-12-28 to 2025-01-03
26-week rolling beta date range: 2018-06-29 to 2025-01-03

Missing beta in rolling_52: 0
Missing beta in rolling_26: 0
```

2. Define crisis windows

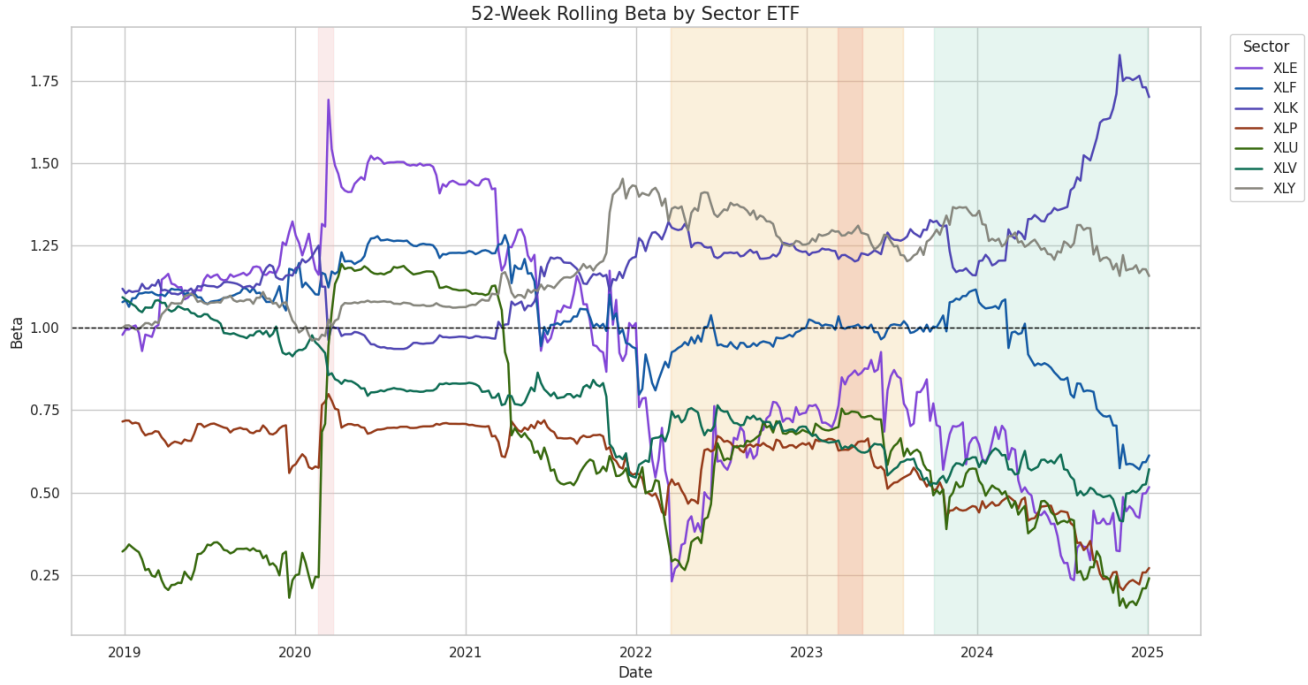
```
SECTOR_COLORS = {
    "XLK": "#534AB7",
    "XLE": "#854ADB",
    "XLF": "#185EA5",
    "XLV": "#0F6E56",
    "XLP": "#993C1D",
    "XLU": "#3B6D11",
    "XLY": "#888780",
}

EVENT_WINDOWS = [
    ("COVID Crash", pd.Timestamp("2020-02-19"), pd.Timestamp("2020-03-23"), "#f7c1c1"),
    ("Rate Hike", pd.Timestamp("2022-03-16"), pd.Timestamp("2023-07-26"), "#fac775"),
    ("SVB Crisis", pd.Timestamp("2023-03-08"), pd.Timestamp("2023-05-01"), "#f0997b"),
    ("AI Rally", pd.Timestamp("2023-10-01"), pd.Timestamp("2024-12-31"), "#9FE1CB"),
]

print("\nEvent window row counts:")
for label, start, end, color in EVENT_WINDOWS:
    temp = rolling_52[(rolling_52["date"] >= start) & (rolling_52["date"] <= end)]
    print(label, temp.shape[0])
```

```
Event window row counts:
COVID Crash 35
Rate Hike 497
SVB Crisis 56
AI Rally 455
```

3. Rolling beta line chart

- This figure shows the time-varying beta of each sector ETF relative to the market.

```
plt.figure(figsize=(15, 8))

for sector in sector_order:
    temp = rolling_52[rolling_52["sector"] == sector].sort_values("date")
    plt.plot(
        temp["date"],
        temp["beta"],
        label=sector,
        linewidth=1.8,
        color=SECTOR_COLORS.get(sector, None)
    )

plt.axhline(y=1.0, linestyle="--", linewidth=1, color="black")

for label, start, end, color in EVENT_WINDOWS:
    plt.axvspan(start, end, alpha=0.25, color=color)

plt.title("52-Week Rolling Beta by Sector ETF", fontsize=15)
plt.xlabel("Date")
plt.ylabel("Beta")
plt.legend(title="Sector", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()

plt.savefig(FIGURE_OUT_DIR / "rolling_beta_linechart.png", dpi=300, bbox_inches="tight")
plt.show()
```



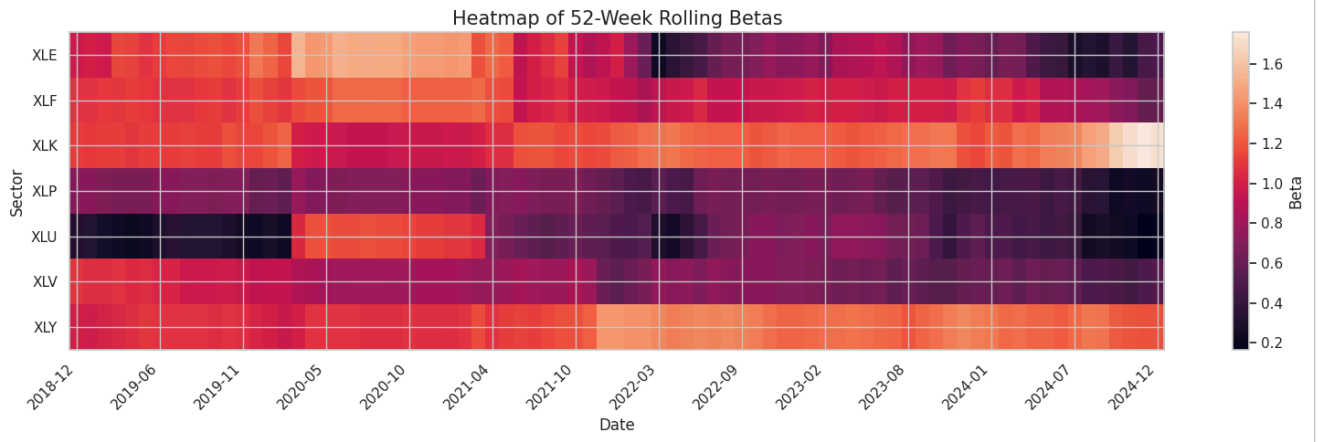
Interpretation

- Betas are clearly time-varying rather than constant.
- Some sectors consistently exhibit higher market sensitivity.
- Defensive sectors tend to remain below 1.
- Betas change significantly during major market events, suggesting strong regime dependence.

4. Beta heatmap

- This heatmap provides a compact view of how sector betas evolve over time.

```
heatmap_df = rolling_52.pivot(index="sector", columns="date", values="beta")
heatmap_df = heatmap_df.loc[sector_order]
heatmap_small = heatmap_df.iloc[:, ::4]

fig, ax = plt.subplots(figsize=(16, 5))
im = ax.imshow(heatmap_small.values, aspect="auto")

ax.set_yticks(np.arange(len(heatmap_small.index)))
ax.set_yticklabels(heatmap_small.index)

x_positions = np.arange(len(heatmap_small.columns))
x_labels = [d.strftime("%Y-%m") for d in heatmap_small.columns]

ax.set_xticks(x_positions[::6])
ax.set_xticklabels(np.array(x_labels)[::6], rotation=45, ha="right")

ax.set_title("Heatmap of 52-Week Rolling Betas", fontsize=15)
ax.set_xlabel("Date")
ax.set_ylabel("Sector")

cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Beta")

plt.tight_layout()
plt.savefig(FIGURE_OUT_DIR / "rolling_beta_heatmap.png", dpi=300, bbox_inches="tight")
plt.show()
```



Interpretation

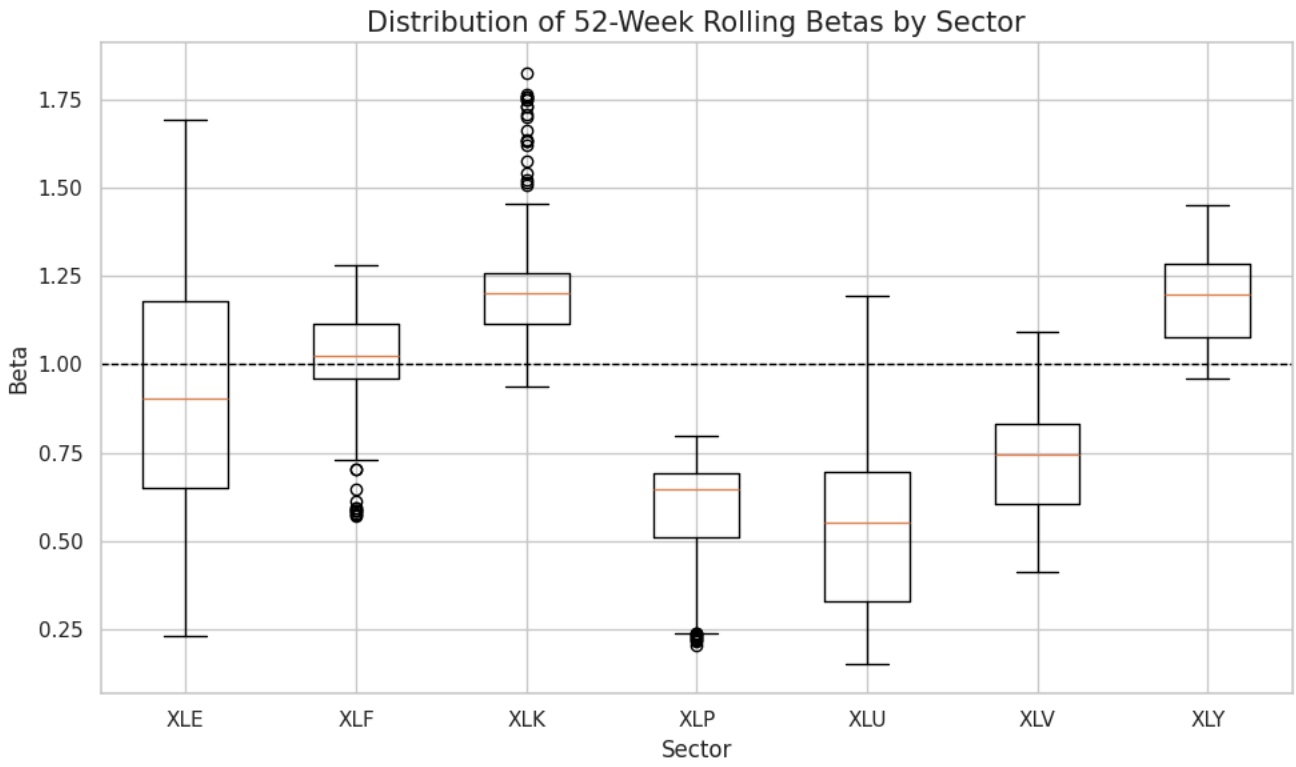
- Darker colors indicate higher beta values.
- Periods of elevated beta often occur simultaneously across multiple sectors.
- Some sectors display persistently higher sensitivity to market movements.
- The heatmap highlights the time-varying structure of systematic risk.

5. Beta boxplot

- This plot compares the distribution of rolling betas across sectors.

```
box_data = [
    rolling_52.loc[rolling_52["sector"] == sector, "beta"].dropna()
    for sector in sector_order
]

plt.figure(figsize=(10, 6))
plt.boxplot(box_data, tick_labels=sector_order, patch_artist=False)
plt.axhline(y=1.0, linestyle="--", linewidth=1, color="black")

plt.title("Distribution of 52-Week Rolling Betas by Sector", fontsize=15)
```

```
plt.xlabel("Sector")
plt.ylabel("Beta")

plt.tight_layout()
plt.savefig(FIGURE_OUT_DIR / "rolling_beta_boxplot.png", dpi=300, bbox_inches="tight")
plt.show()
```



Interpretation

- Sectors differ significantly in their average beta levels.
- Some sectors have wider distributions, indicating unstable exposure.
- Defensive sectors show tighter distributions and lower medians.
- Cyclical sectors tend to have higher and more volatile betas.

6. Crisis bar chart

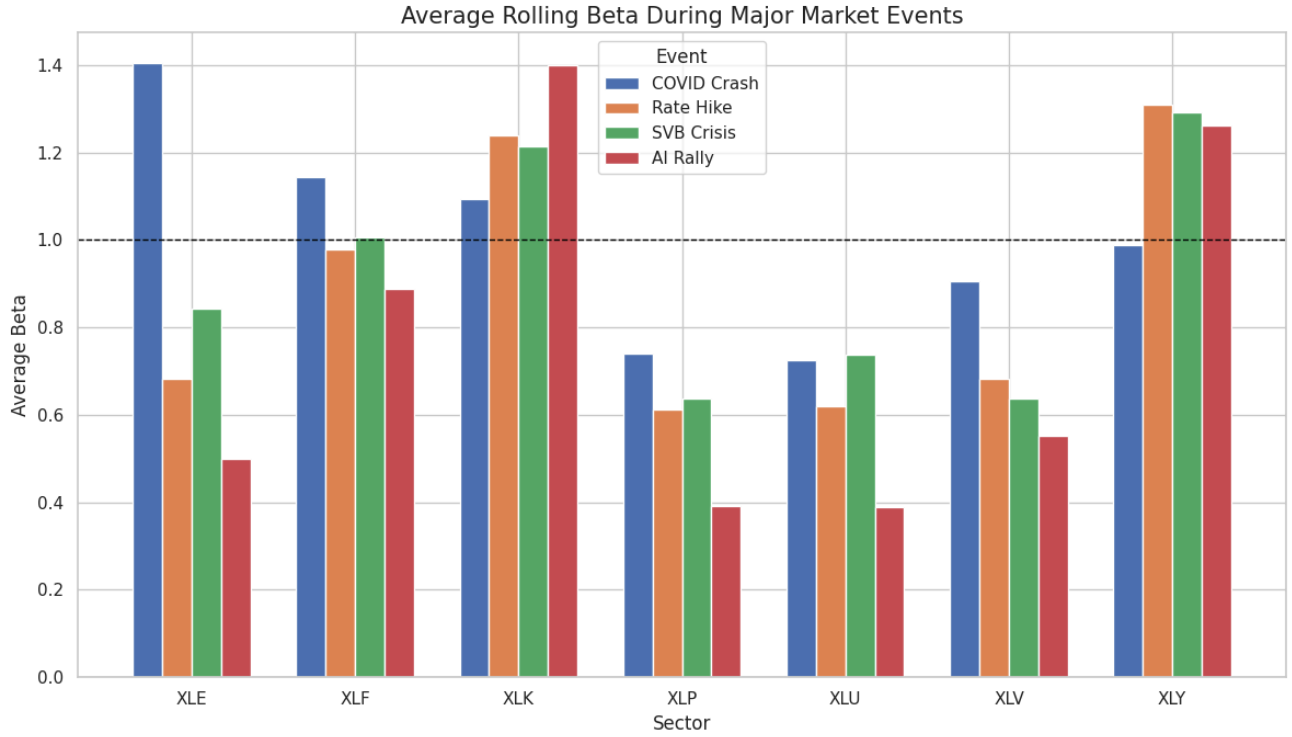
- This chart compares sector betas during major market events.

```
summary_list = []

for label, start, end, color in EVENT_WINDOWS:
    temp = rolling_52[
        (rolling_52["date"] >= start) &
        (rolling_52["date"] <= end)
    ]

    grouped = temp.groupby("sector", as_index=False)["beta"].mean()
    grouped["event"] = label
    summary_list.append(grouped)

crisis_summary = pd.concat(summary_list, ignore_index=True)
crisis_summary = crisis_summary[["event", "sector", "beta"]]
crisis_summary = crisis_summary.rename(columns={"beta": "mean_beta"})

crisis_summary.to_csv(
    FIGURE_OUT_DIR / "crisis_beta_summary.csv",
    index=False
)

print("\nCrisis beta summary:")
```

```
display(crisis_summary.head(12))

pivot_summary = crisis_summary.pivot(
    index="sector",
    columns="event",
    values="mean_beta"
)

pivot_summary = pivot_summary.loc[sector_order]

x = np.arange(len(pivot_summary.index))
bar_width = 0.18
event_order = [e[0] for e in EVENT_WINDOWS]

plt.figure(figsize=(12, 7))

for i, event in enumerate(event_order):
    plt.bar(
        x + i * bar_width,
        pivot_summary[event].values,
        width=bar_width,
        label=event
    )

plt.xticks(
    x + bar_width * (len(event_order) - 1) / 2,
    pivot_summary.index
)

plt.axhline(y=1.0, linestyle="--", linewidth=1, color="black")

plt.title("Average Rolling Beta During Major Market Events", fontsize=15)
plt.xlabel("Sector")
plt.ylabel("Average Beta")
plt.legend(title="Event")

plt.tight_layout()
plt.savefig(FIGURE_OUT_DIR / "rolling_beta_crisis_bars.png", dpi=300, bbox_inches="tight")
plt.show()
```

```
Crisis beta summary:
```

	event	sector	mean_beta
0	COVID Crash	XLE	1.403507
1	COVID Crash	XLF	1.144621
2	COVID Crash	XLK	1.093493
3	COVID Crash	XLP	0.739054
4	COVID Crash	XLU	0.725704
5	COVID Crash	XLV	0.904844
6	COVID Crash	XLY	0.989124
7	Rate Hike	XLE	0.681402
8	Rate Hike	XLF	0.979212
9	Rate Hike	XLK	1.238676
10	Rate Hike	XLP	0.612469
11	Rate Hike	XLU	0.618885



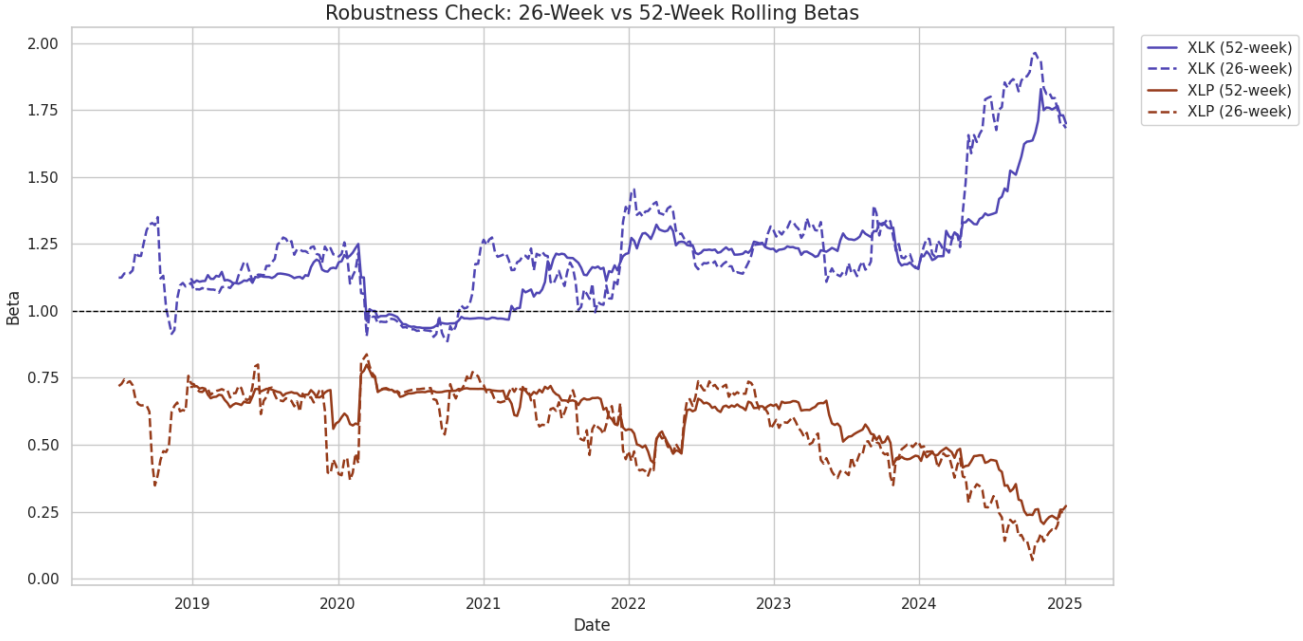
Interpretation

- Sector betas shift noticeably during crisis periods.
- Relative ranking of sectors can change under stress.
- Some sectors become more sensitive to market shocks.
- This supports the idea that beta depends on market regimes.

7. Robustness check: 26-week vs 52-week

- This plot compares rolling beta estimates using different window lengths.

```
robust_52 = rolling_52[rolling_52["sector"].isin(["XLK", "XLP"])].copy()
robust_52["window_label"] = "52-week"

robust_26 = rolling_26[rolling_26["sector"].isin(["XLK", "XLP"])].copy()
robust_26["window_label"] = "26-week"

robust_all = pd.concat([robust_52, robust_26], ignore_index=True)

plt.figure(figsize=(14, 7))

for sector in ["XLK", "XLP"]:
    for window_label in ["52-week", "26-week"]:
        temp = robust_all[
            (robust_all["sector"] == sector) &
            (robust_all["window_label"] == window_label)
        ].sort_values("date")

        line_style = "-" if window_label == "52-week" else "--"

        plt.plot(
            temp["date"],
            temp["beta"],
            linestyle=line_style,
            linewidth=1.8,
            label=f"{sector} ({window_label})",
            color=SECTOR_COLORS.get(sector, None)
        )

plt.axhline(y=1.0, linestyle="--", linewidth=1, color="black")

plt.title("Robustness Check: 26-Week vs 52-Week Rolling Betas", fontsize=15)
plt.xlabel("Date")
plt.ylabel("Beta")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")

plt.tight_layout()
plt.savefig(FIGURE_OUT_DIR / "rolling_beta_robustness_26w.png", dpi=300, bbox_inches="tight")
plt.show()
```



Interpretation

- Shorter windows produce more responsive beta estimates.

- Longer windows provide smoother trends.
- Despite differences in volatility, the overall patterns remain consistent.
- This suggests the results are robust to window selection.

8. Final output check

```
print("\nGenerated files saved to:")
print(FIGURE_OUT_DIR)

print("\nGenerated files:")
for file in sorted(FIGURE_OUT_DIR.iterdir()):
    print("-", file.name)
```

```
Generated files saved to:
/content/drive/MyDrive/5261 Final Project/output/figures

Generated files:
- crisis_beta_summary.csv
- rolling_beta_boxplot.png
- rolling_beta_crisis_bars.png
- rolling_beta_heatmap.png
- rolling_beta_linechart.png
- rolling_beta_robustness_26w.png
```

Summary

- Rolling betas are not constant and vary over time.
- There are clear differences between cyclical and defensive sectors.
- Betas respond strongly to major market events.
- The findings support the idea that market exposure is regime-dependent.

6. Regime and Crisis Analysis

1. Load Data

```
# DATA_DIR and OUT_DIR are defined in the setup cell.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
import warnings
warnings.filterwarnings("ignore")

OUT_DIR.mkdir(parents=True, exist_ok=True)
FIGURE_OUT_DIR.mkdir(parents=True, exist_ok=True)
```

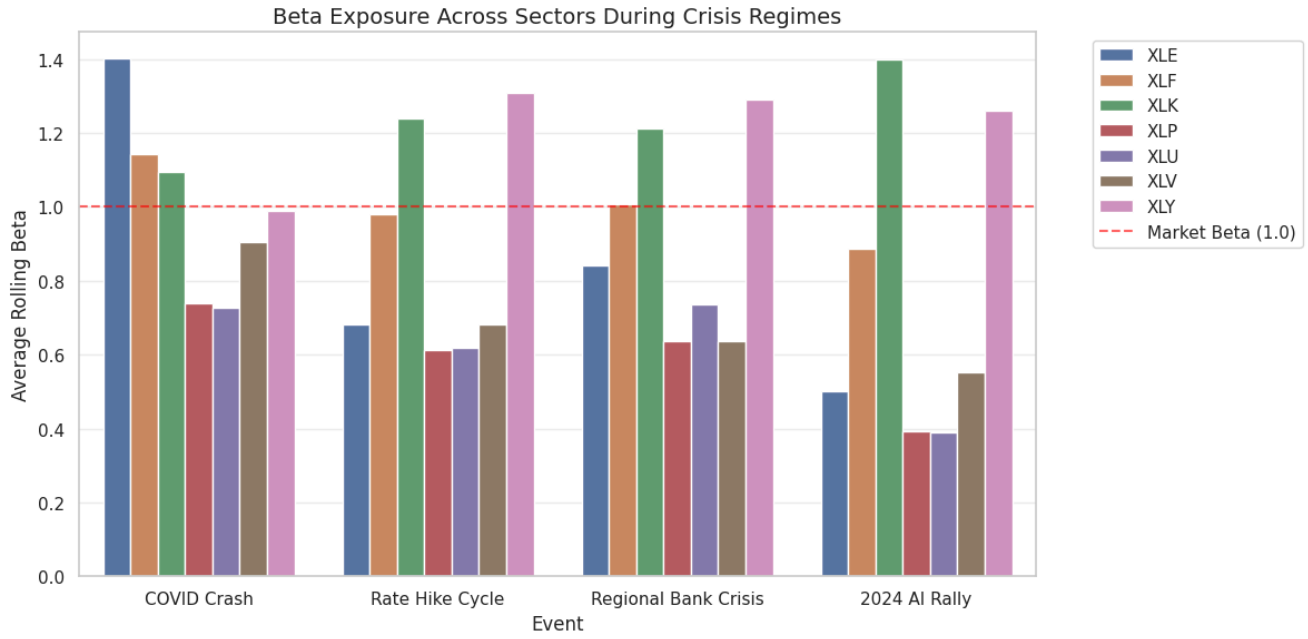
This section performs a detailed crisis analysis by examining how sector betas change during predefined crisis events. It calculates and visualizes the average rolling betas before, during, and after specific market events to understand how different sectors react to stress periods.

```
rolling_52 = pd.read_csv(
    OUT_DIR / "rolling_betas.csv",
    parse_dates=["date"]
)

rolling_26 = pd.read_csv(
    OUT_DIR / "rolling_betas_26w.csv",
    parse_dates=["date"]
)

weekly = pd.read_csv(
    OUT_DIR / "returns_clean_weekly.csv",
    parse_dates=["date"]
```

```
)

rolling_52 = rolling_52.sort_values(["sector", "date"]).reset_index(drop=True)
rolling_26 = rolling_26.sort_values(["sector", "date"]).reset_index(drop=True)
weekly = weekly.sort_values("date").reset_index(drop=True)

sector_order = sorted(rolling_52["sector"].unique())
```

2. Crisis Analysis

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

crisis_events = {
    "COVID Crash": ("2020-02-19", "2020-03-23"),
    "Rate Hike Cycle": ("2022-03-16", "2023-07-26"),
    "Regional Bank Crisis": ("2023-03-08", "2023-05-01"),
    "2024 AI Rally": ("2023-10-01", "2024-12-31")
}

results = []
sectors = sorted(rolling_52['sector'].unique())

for event_name, (start_date, end_date) in crisis_events.items():
    start_dt = pd.to_datetime(start_date)
    end_dt = pd.to_datetime(end_date)

    pre_start = start_dt - pd.DateOffset(months=3)
    post_end = end_dt + pd.DateOffset(months=3)

    for sector in sectors:
        sector_data = rolling_52[rolling_52['sector'] == sector].copy()

        avg_pre = sector_data[(sector_data['date'] >= pre_start) & (sector_data['date'] < start_dt)]['beta'].mean()
        avg_during = sector_data[(sector_data['date'] >= start_dt) & (sector_data['date'] <= end_dt)]['beta'].mean()
        avg_post = sector_data[(sector_data['date'] > end_dt) & (sector_data['date'] <= post_end)]['beta'].mean()

        results.append({
            'Event': event_name,
            'Sector': sector,
            'Pre-Crisis Beta': avg_pre,
            'During-Crisis Beta': avg_during,
            'Post-Crisis Beta': avg_post,
            'Beta_Jump': avg_during - avg_pre if avg_pre is not None else np.nan
        })

crisis_analysis_df = pd.DataFrame(results)

plt.figure(figsize=(12, 6))
sns.barplot(data=crisis_analysis_df, x='Event', y='During-Crisis Beta', hue='Sector')
plt.axhline(1.0, color='red', linestyle='--', alpha=0.6, label='Market Beta (1.0)')
plt.title('Beta Exposure Across Sectors During Crisis Regimes', fontsize=14)
plt.ylabel('Average Rolling Beta')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig(OUT_DIR / "crisis_beta_exposure.png", dpi=300, bbox_inches="tight")
plt.show()
```



Crisis Beta Summary Table

```
print("--- Crisis Beta Summary Table ---")
summary_table = crisis_analysis_df.pivot(index='Sector', columns='Event', values='During-Crisis Beta').round(3)
print(summary_table)
```

```
--- Crisis Beta Summary Table ---
Event   2024 AI Rally  COVID Crash  Rate Hike Cycle  Regional Bank Crisis
Sector
XLE             0.501        1.404            0.681                 0.843
XLF             0.888        1.145            0.979                 1.007
XLK             1.400        1.093            1.239                 1.213
XLP             0.391        0.739            0.612                 0.638
XLU             0.390        0.726            0.619                 0.737
XLV             0.552        0.905            0.683                 0.637
XLY             1.262        0.989            1.308                 1.290
```

Beta Stability (Standard Deviation of Rolling Beta)

```
beta_stability = rolling_52.groupby('sector')['beta'].std().sort_values()
print("\n--- Beta Stability (Standard Deviation of Rolling Beta) ---")
print(beta_stability)
```

```
--- Beta Stability (Standard Deviation of Rolling Beta) ---
sector
XLY    0.124440
XLP    0.128926
XLF    0.148493
XLV    0.166186
XLK    0.172027
XLU    0.298897
XLE    0.362159
Name: beta, dtype: float64
```

Explanation of the Crisis Analysis

1. **Crisis Event Definition:**
   - The `crisis_events` dictionary defines the start and end dates for several key market events, such as 'COVID Crash', 'Rate Hike Cycle', 'Regional Bank Crisis', and '2024 AI Rally'.

2. **Beta Calculation for Crisis Periods:**
    - For each crisis event and each sector, the code calculates the average rolling beta during three periods:
        - **Pre-Crisis:** The 3 months leading up to the crisis start date.
        - **During-Crisis:** The period between the crisis start and end dates.
        - **Post-Crisis:** The 3 months immediately following the crisis end date.
    - This is done by filtering the `rolling_52` DataFrame (which contains 52-week rolling beta estimates) based on the date ranges for each period and then computing the mean beta.
3. **Beta Jump Calculation:**
    - A 'Beta_Jump' metric is calculated as the difference between the 'During-Crisis Beta' and the 'Pre-Crisis Beta'. This helps quantify the immediate change in a sector's market sensitivity during a crisis.
4. **Visualization (Bar Plot):**
    - A bar plot is generated using `seaborn` to visually compare the 'During-Crisis Beta' across different sectors for each crisis event.
    - A horizontal dashed line at 1.0 is added to represent the market's beta, providing a clear reference point.
    - The plot helps to quickly identify which sectors become more or less sensitive to market movements during each crisis.
5. **Summary Table:**
    - The `crisis_analysis_df` is created to store all the calculated pre-, during-, and post-crisis betas, along with the beta jump, for each sector and event.
    - A pivoted summary table (`summary_table`) is printed, showing the 'During-Crisis Beta' for each sector across all defined crisis events in a more readable format.
6. **Beta Stability:**
    - The standard deviation of the rolling beta for each sector (`beta_stability`) is calculated and printed. This provides an overall measure of how volatile or stable a sector's beta has been throughout the entire sample period, indicating how much its market sensitivity fluctuates over time.

7. Predictive Extension

This section tests whether end-of-month 52-week rolling beta predicts next-month realized sector volatility. A positive and statistically significant slope indicates that rolling beta carries forward-looking information rather than only summarizing historical co-movement.

Setup & Load Data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')

# DATA_DIR and OUT_DIR are defined in the setup cell.
OUT_DIR.mkdir(parents=True, exist_ok=True)

SECTORS = ['XLK', 'XLE', 'XLF', 'XLV', 'XLP', 'XLU', 'XLY']

SECTOR_COLORS = {
    'XLK': '#534AB7', 'XLE': '#854ADB', 'XLF': '#185EA5',
    'XLV': '#0F6E56', 'XLP': '#993C1D', 'XLU': '#3B6D11', 'XLY': '#888780',
}

plt.rcParams.update({
    'figure.facecolor': 'white', 'axes.facecolor': 'white',
    'axes.grid': True, 'grid.alpha': 0.3,
    'axes.spines.top': False, 'axes.spines.right': False,
})

betas   = pd.read_csv(OUT_DIR / 'rolling_betas.csv', parse_dates=['date'])
betas   = betas[betas['window'] == 52].copy()
returns = pd.read_csv(OUT_DIR / 'returns_clean.csv', parse_dates=['date'])
```

```
print(f'Rolling betas shape : {betas.shape}')
print(f'Returns shape       : {returns.shape}')
```

```
Rolling betas shape : (2205, 7)
Returns shape       : (1760, 33)
```

2. Construct Predictor and Target Variables

- **Predictor β_t**: end-of-month rolling beta for each sector
- **Target**: next-month realized volatility = `std(daily returns in month t+1) × √21`
- Merged via `shift(−1)` so beta at month *t* predicts volatility at month *t+1*

```
# End-of-month beta per sector
betas['ym'] = betas['date'].dt.to_period('M')
beta_monthly = (
    betas.sort_values('date')
         .groupby(['ym', 'sector']).last()
         .reset_index()[['ym', 'sector', 'beta']]
)

# Next-month realized volatility
returns['ym'] = returns['date'].dt.to_period('M')
monthly_vol = pd.concat([
    returns.groupby('ym')[f'ret_{s}'].std().mul(np.sqrt(21))
           .rename('realized_vol').reset_index().assign(sector=s)
    for s in SECTORS
])

# Merge and shift: beta_t → vol_{t+1}
df_ext = beta_monthly.merge(monthly_vol, on=['ym', 'sector'])
df_ext = df_ext.sort_values(['sector', 'ym'])
df_ext['vol_next'] = df_ext.groupby('sector')['realized_vol'].shift(-1)
df_ext = df_ext.dropna(subset=['vol_next', 'beta'])

print(f'Observations: {len(df_ext)}  |  Months per sector: {len(df_ext) // len(SECTORS)}')
df_ext.groupby('sector')[['beta', 'vol_next']].mean().round(4)
```

```
Observations: 504  |  Months per sector: 72
```

sector	beta	vol_next
XLE	0.9298	0.0838
XLF	1.0417	0.0587
XLK	1.1887	0.0677
XLP	0.5978	0.0381
XLU	0.5952	0.0521
XLV	0.7524	0.0429
XLY	1.1914	0.0620

3. OLS Regression: beta_t → realized_vol_{t+1}

Run per-sector and pooled OLS. Key test: is **b > 0** and statistically significant?

```
results = []
for s in SECTORS:
    sub = df_ext[df_ext['sector'] == s]
    m   = sm.OLS(sub['vol_next'], sm.add_constant(sub['beta'])).fit()
    p   = m.pvalues['beta']
    results.append({'Sector': s,
                    'b (coef)': round(m.params['beta'], 4),
                    't-stat':   round(m.tvalues['beta'], 3),
                    'p-value':  round(p, 4),
                    'R²':       round(m.rsquared, 4),
                    'N':        int(m.nobs),
                    'Sig':      '***' if p<0.01 else '**' if p<0.05 else '*' if p<0.10 else 'n.s.'})
```

```
m_pool = sm.OLS(df_ext['vol_next'], sm.add_constant(df_ext['beta'])).fit()
p_pool = m_pool.pvalues['beta']
results.append({'Sector': 'Pooled',
                'b (coef)': round(m_pool.params['beta'], 4),
                't-stat':   round(m_pool.tvalues['beta'], 3),
                'p-value':  round(p_pool, 4),
                'R²':       round(m_pool.rsquared, 4),
                'N':        int(m_pool.nobs),
                'Sig':      '***' if p_pool<0.01 else '**' if p_pool<0.05 else '*' if p_pool<0.10 else 'n.s.'})

reg_df = pd.DataFrame(results)
reg_df.to_csv(OUT_DIR / 'extension_regression_table.csv', index=False)
```

```
Sector  b (coef)  t-stat  p-value     R²   N  Sig
   XLK   -0.0084  -0.306   0.7603 0.0013  72 n.s.
   XLE    0.0490   3.039   0.0033 0.1165  72  ***
   XLF    0.0577   1.672   0.0990 0.0384  72    *
   XLV    0.0343   1.793   0.0772 0.0439  72    *
   XLP    0.0414   1.656   0.1021 0.0377  72 n.s.
   XLU    0.0303   2.179   0.0327 0.0635  72   **
   XLY    0.0532   1.652   0.1029 0.0375  72 n.s.
Pooled    0.0380   7.202   0.0000 0.0936 504  ***
```

4. Scatter Plot: Rolling Beta vs Next-Month Realized Volatility

```
fig, axes = plt.subplots(2, 4, figsize=(18, 9))

for i, s in enumerate(SECTORS):
    ax  = axes.flatten()[i]
    sub = df_ext[df_ext['sector'] == s]
    row = reg_df[reg_df['Sector'] == s].iloc[0]
    m_s = sm.OLS(sub['vol_next'], sm.add_constant(sub['beta'])).fit()
    x   = np.linspace(sub['beta'].min(), sub['beta'].max(), 100)

    ax.scatter(sub['beta'], sub['vol_next'],
               color=SECTOR_COLORS[s], alpha=0.5, s=30, zorder=3)
    ax.plot(x, m_s.params['const'] + m_s.params['beta'] * x,
            color='black', lw=1.8, zorder=4)
    ax.annotate(
        f"b = {row['b (coef)']:.3f}  {row['Sig']}\nt = {row['t-stat']:.2f}  R² = {row['R²']:.3f}",
        xy=(0.05, 0.75), xycoords='axes fraction', fontsize=9,
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.85)
    )
    ax.set_title(s, fontsize=13, fontweight='bold', color=SECTOR_COLORS[s])
    ax.set_xlabel('Rolling Beta (end of month t)')
    ax.set_ylabel('Realized Volatility (month t+1)')

axes.flatten()[-1].set_visible(False)
fig.suptitle('Extension: Can Rolling Beta Predict Next-Month Sector Volatility?',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(OUT_DIR / 'extension_scatter.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: extension_scatter.png')
```